



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Technischer Entwurf

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 10. Juli 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Lucas Neidahl ,Luan Hyseni, Marvin Voigt, Mohammed Alhalabi,	aus dem Fachentwurf (angepasst)
1.1	Luan Hyseni, Lucas Neidahl, Marvin Voigt	aus dem Fachentwurf (angepasst)
2	Lucas Neidahl	aus dem Fachentwurf
2.1	Lucas Neidahl	aus dem Fachentwurf
2.2	Lucas Neidahl	aus dem Fachentwurf
2.3	Lucas Neidahl	aus dem Fachentwurf
2.4	Lucas Neidahl	aus dem Fachentwurf
2.5	Lucas Neidahl	aus dem Fachentwurf
2.6	Lucas Neidahl	aus dem Fachentwurf
2.7	Mohammed Alhalabi	aus dem Fachentwurf
2.8	Mohammed Alhalabi	aus dem Fachentwurf
3	Lucas Neidahl, Marvin Voigt	fertig
3.1	Lucas Neidahl, Marvin Voigt	fertig
3.2	Lucas Neidahl, Marvin Voigt	fertig
3.3	Lucas Neidahl, Marvin Voigt	fertig
4	Luan Hyseni	fertig
5	Luan Hyseni , Mohamed Sabri Jlizi, Mohammad Khatib	fertig
5.1	Mohammad Khatib , Mohamed Sabri Jlizi	fertig
5.2	Luan Hyseni	fertig
5.3	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
5.4	Mohamed Sabri Jlizi, Mohammad Khatib	fertig

5.5	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
5.6	Luan Hyseni	fertig
5.7	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
5.8	Luan Hyseni	
6	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
6.1	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
6.2	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
7	Marvin Voigt	aus dem Fachentwurf
7.1	Marvin Voigt	aus dem Fachentwurf
7.2	Marvin Voigt	aus dem Fachentwurf
7.3	Marvin Voigt	aus dem Fachentwurf
7.4	Marvin Voigt, Moham- med Alhalabi	aus dem Fachentwurf
7.5	Marvin Voigt	aus dem Fachentwurf
7.6	Marvin Voigt, Moham- med Alhalabi	aus dem Fachentwurf
8	Mohammed Alhalabi	fertig
9	Mohammed Alhalabi	fertig
9.1	Mohammed Alhalabi	fertig
9.2	Mohammed Alhalabi	fertig
9.3	Mohammed Alhalabi	fertig
10	Mohammed Alhalabi	fertig

Inhaltsverzeichnis

1	Einleitung	7
1.1	Projektübersicht	7
1.2	Projektdetails	8
1.2.1	Szenenkonfiguration	8
1.2.2	Drohne (Drone Lifecycle)	10
1.2.3	Aktivitätsdiagramm für den Workflow Bild zu Formation	11
2	Analyse der Produktfunktionen	12
2.1	Analyse von Funktion F10: Konfiguration der Show	12
2.2	Analyse von Funktion F11: Szene anlegen und Metadaten erfassen	13
2.3	Analyse von Funktion F12: Formation und Bewegung definieren	14
2.4	Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren	15
2.5	Analyse von Funktion F14: Szene speichern und zur Simulation übergeben	16
2.6	Analyse von Funktion F20: Choreographie simulieren	17
2.7	Analyse von Funktion F30: Projekt verwalten	19
2.8	Analyse von Funktion F40: Szenenprüfung auf Eingabefehler	20
3	Resultierende Softwarearchitektur	22
3.1	Komponentenspezifikation	22
3.2	Schnittstellenspezifikation	24
3.3	Protokolle für die Benutzung der Komponenten	26
3.3.1	GUI	26
3.3.2	ShowKonfigurator	28
3.3.3	Projektmanagement-Modul	30
3.3.4	Simulationsbibliothek	32
4	Verteilungsentwurf	34
5	Implementierungsentwurf	35
5.1	Implementierung von Komponente C10 – GUI	35
5.1.1	Paket-/Klassendiagramm	36
5.1.2	Erläuterung	36

5.2	C20 – ShowKonfigurator:	41
5.2.1	Paket-/Klassendiagramm	41
5.3	C30 – Projektmanagement-Modul:	46
5.3.1	Paket-/Klassendiagramm	46
5.4	C40 – FormationsBibliothek:	51
5.4.1	Paket-/Klassendiagramm	51
5.5	C50 – LightEffectLibrary:	54
5.5.1	Paket-/Klassendiagramm	54
5.6	C70 – Validierungskomponente	57
5.6.1	Paket-/Klassendiagramm	58
5.7	C80 – Datensysteme:	60
5.7.1	Paket-/Klassendiagramm	61
5.8	C90 – Szenenverwaltung	63
5.8.1	Paket-/Klassendiagramm	64
6	Datenmodell	66
6.1	Diagramm	66
6.2	Erläuterung	67
7	Konfiguration	71
7.1	Technische Voraussetzungen	71
7.2	Projektstruktur	72
7.3	Start und Nutzung der Anwendung	72
7.4	Konfigurationsdateien	73
7.5	Konfiguration über die Benutzeroberfläche	74
7.6	Mögliche Erweiterungen	74
8	Änderungen gegenüber Fachentwurf	75
9	Erfüllung der Kriterien	77
9.1	Musskriterien	77
9.2	Sollkriterien	78
9.3	Kannkriterien	78
10	Glossar	79

Abbildungsverzeichnis

1.1	Ein Aktivitätsdiagramm für die Software	7
1.2	Zustandsdiagramm der Szenenkonfiguration	8
1.3	Zustandsdiagramm zum Drohnenlebenszyklus	10
1.4	Aktivitätsdiagramm der Bild zu Coordinate Funktion	11
2.1	Sequenzdiagramm für F10	13
2.2	Sequenzdiagramm für F11	14
2.3	Sequenzdiagramm für F12	15
2.4	Sequenzdiagramm für F13	16
2.5	Sequenzdiagramm für F14	17
2.6	Sequenzdiagramm für F20	18
2.7	Sequenzdiagramm für F30	20
2.8	Sequenzdiagramm für F40	21
3.1	Komponentendiagramm	22
3.2	Zustandsdiagramm: GUI	26
3.3	Zustandsdiagramm: ShowKonfigurator	28
3.4	Zustandsdiagramm: Projektmanagment-Modul	30
3.5	Zustandsdiagramm: Simulationsbibliothek	32
5.1	Klassendiagramm für Komponente $\langle C10 \rangle$	36
5.2	Klassendiagramm für Komponente ShowKonfigurator (C20)	42
5.3	Klassendiagramm für Komponente $\langle C30 \rangle$	46
5.4	Klassendiagramm für Komponente $\langle C40 \rangle$	51
5.5	Klassendiagramm für Komponente $\langle C40 \rangle$	54
5.6	Klassendiagramm für Komponente C70	58
5.7	Klassendiagramm für Komponente $\langle C80 \rangle$	61
5.8	Klassendiagramm für Komponente Szenenverwaltung (C90)	64
6.1	Klassendiagramm Produktdaten	66

1 Einleitung

Dieses Dokument beschreibt den Technischen Entwurf der Software SkyOrchestrator, die im Rahmen des Softwareentwicklungspraktikums im Sommersemester 2025 an der Technischen Universität Braunschweig entwickelt wird. Ziel ist es aufbauend auf dem Fachentwurf, die wesentlichen Funktionen und Abläufe der Software detailliert darzustellen und die zugrundeliegenden technischen Entscheidungen nachvollziehbar zu dokumentieren. Damit wird eine klare Grundlage für die Implementierung und Weiterentwicklung geschaffen.

Im ersten Abschnitt wird die Arbeitsweise des Systems anhand eines Aktivitätsdiagramms visualisiert und erläutert, um die Benutzerinteraktionen und Systemzustände anschaulich darzustellen. Anschließend werden die einzelnen Komponenten und deren interne Zustände anhand von Zustandsdiagrammen beschrieben, die den Lebenszyklus der Szenenkonfiguration sowie der Drohnen im System abbilden.

1.1 Projektübersicht

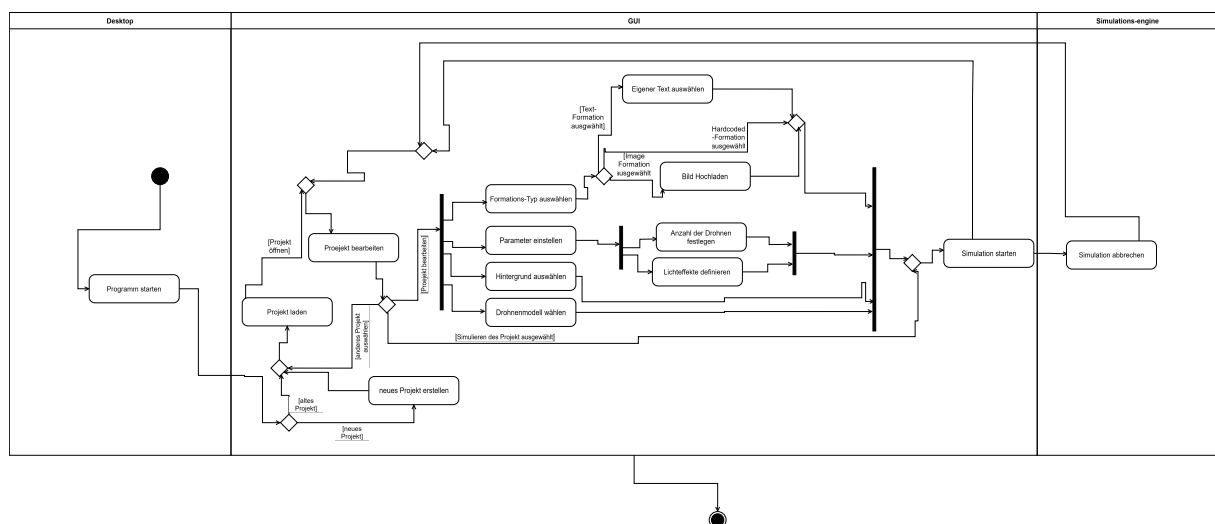


Abbildung 1.1: Ein Aktivitätsdiagramm für die Software

Der Nutzer startet die Anwendung auf seinem Desktop. Wenn die Anwendung geladen ist, hat der Nutzer die Möglichkeit, ein Projekt zu laden oder ein neues zu erstellen. Nachdem

er ein neues oder ein altes Projekt geladen hat, kann er dieses nun bearbeiten, ein anderes Projekt auswählen oder eine Simulation starten. Wenn er ein Projekt bearbeiten will, kommt er in den Bearbeitungsmodus und kann verschiedene Parameter für die Simulation festlegen, wie etwa den Formations-Typ, den Hintergrund wählen, das Drohnenmodell auswählen, Bilder hochladen, die Anzahl der Drohnen festlegen und Lichteffekte definieren. Wenn dann alle nötigen Parameter festgelegt wurden, kann die Simulation gestartet werden. Der Nutzer kann auch die Simulation wieder abbrechen und das Projekt wieder bearbeiten oder ein anderes Projekt auswählen. Jederzeit kann das Programm auch beendet werden.

1.2 Projektdetails

1.2.1 Szenenkonfiguration

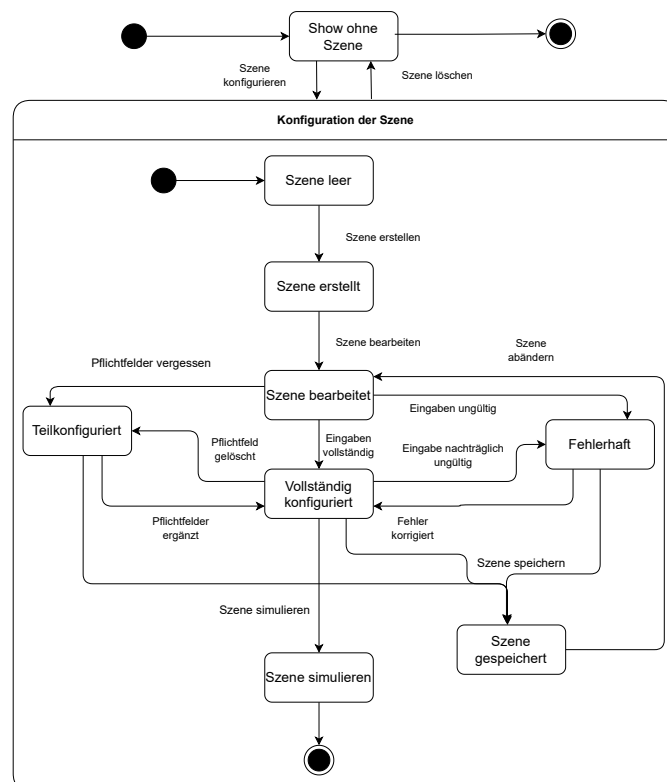


Abbildung 1.2: Zustandsdiagramm der Szenenkonfiguration

Das zweite Zustandsdiagramm stellt den internen Ablauf innerhalb der Szenenkonfiguration dar. Es beschreibt detailliert die möglichen Zustände einer Szene von der Erstellung bis zur Simulation. Der Prozess beginnt mit dem Zustand Szene leer, aus dem heraus eine neue Szene erstellt wird. Nach der Erstellung befindet sich die Szene im Zustand Szene erstellt und kann dann bearbeitet werden. Im Bearbeitungsprozess unterscheidet das System zwischen gültigen und ungültigen Eingaben. Ungültige Eingaben führen zum Zustand Fehlerhaft, während vollständige Eingaben den Zustand Vollständig konfiguriert erreichen. Eine Szene kann auch nur teilkonfiguriert sein, wenn beispielsweise Pflichtfelder fehlen oder gelöscht wurden. In diesem Fall kann sie nach entsprechender Korrektur erneut bearbeitet oder vervollständigt werden. Szenen können unabhängig vom Konfigurationsstand gespeichert und später weiterbearbeitet werden. Eine Simulation ist nur möglich, wenn alle relevanten Eingaben vollständig und korrekt vorliegen. Auch die Rückführung aus fehlerhaften Zuständen zur Bearbeitung ist explizit vorgesehen und modelliert. Beide Diagramme verdeutlichen, wie der SkyOrchestrator komplexe Abläufe strukturiert und intern abbildet. Sie dienen als Grundlage für die Umsetzung der Benutzerinteraktion sowie der automatischen Validierung innerhalb des Systems. .

1.2.2 Drohne (Drone Lifecycle)

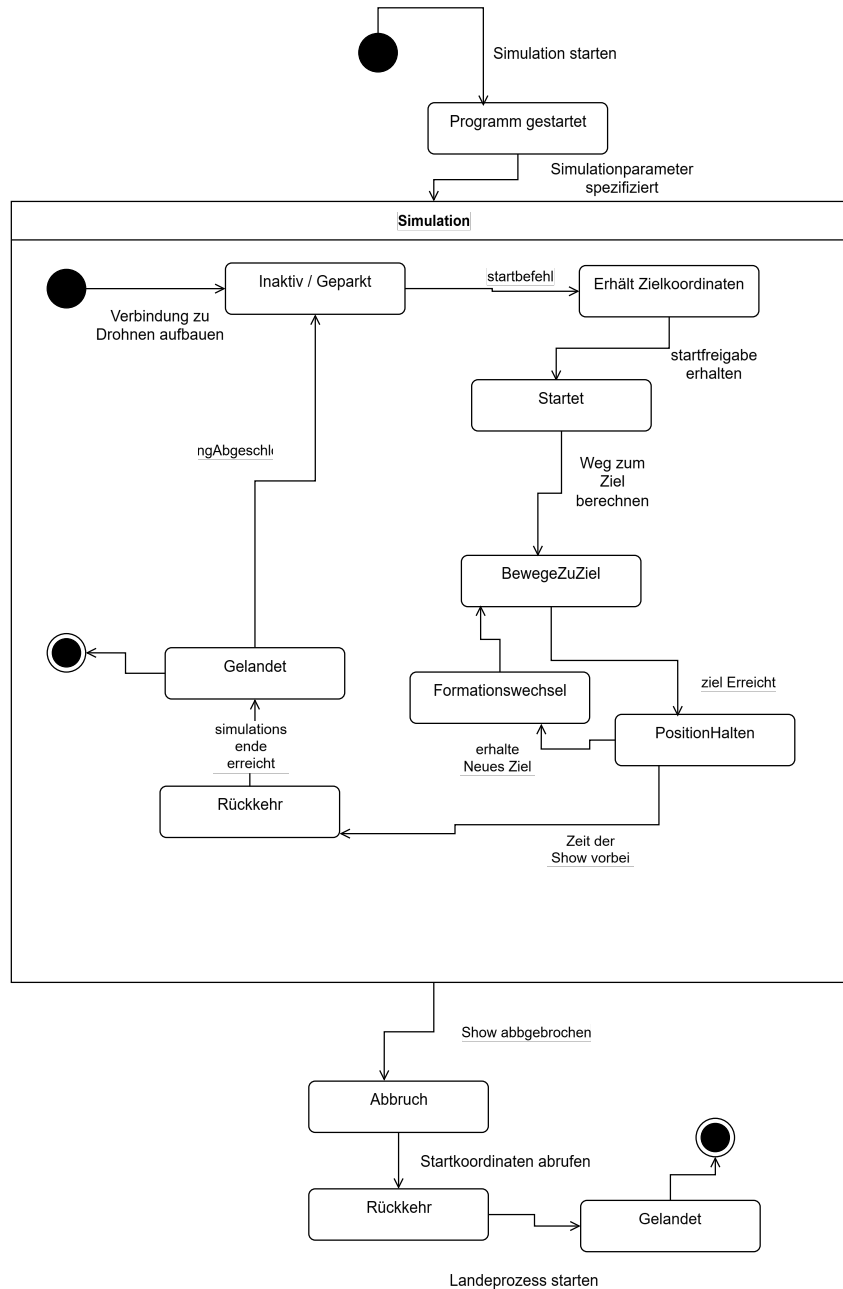


Abbildung 1.3: Zustandsdiagramm zum Drohnenlebenszyklus

Zu Beginn befindet sich die Drohne inaktiv/geparkt und wartet auf einen Startbefehl, woraufhin sie abhebt und auf eine Schwebhöhe steigt (Startet). Anschließend wechselt sie je nach erhaltenen Zielkoordinaten in den Zustand „BewegeZuZiel“ und verharrt nach Erreichen ihrer Position im Zustand „PositionHalten“. Bei einem Formationswechsel erhält sie neue Zielkoordinaten und

wechselt in den Zustand „Formationswechsel“, bis auch hier die Position erreicht ist und sie zurück in „PositionHalten“ geht. Nach dem Simulationsende folgt der Zustand „Rückkehr“, in dem sie zur Startposition zurückkehrt, und schließlich „Gelandet“, sobald die Parkposition erreicht und die Landung abgeschlossen ist. Das Program bzw die Simulation kann auch jederzeit abgebrochen werden, in dem die Drohnen zurückkehren und wieder landen, von ihrer jetzigen Position.

1.2.3 Aktivitätsdiagramm für den Workflow Bild zu Formation

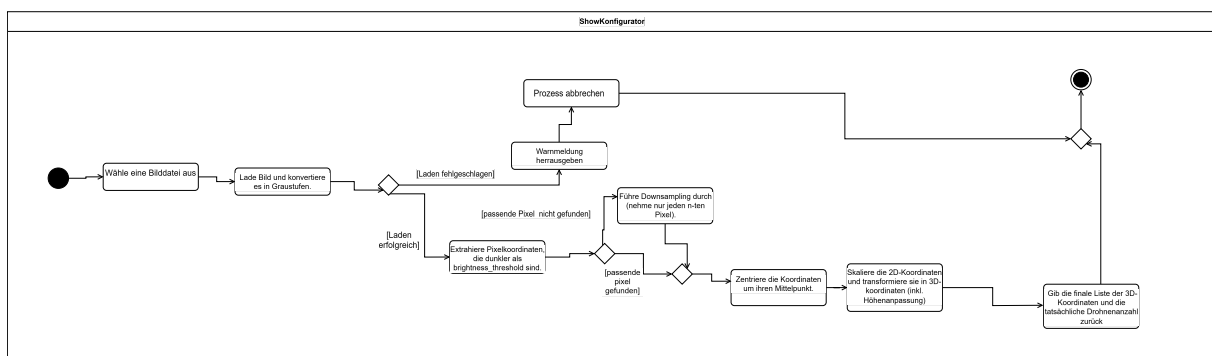


Abbildung 1.4: Aktivitätsdiagramm der Bild zu Coordinate Funktion

Der Workflow beginnt mit der Auswahl einer Bilddatei über einen Dateidialog. Anschließend wird das Bild geladen und in Graustufen konvertiert. Falls das Laden fehlschlägt, wird der Prozess abgebrochen. Andernfalls werden alle Pixel extrahiert, deren Helligkeit unter einem festgelegten Schwellenwert liegt. Wenn dabei keine passenden Pixel gefunden werden, endet der Prozess ebenfalls. Sind jedoch Pixel vorhanden, wird geprüft, ob deren Anzahl die Zielanzahl an Drohnen überschreitet. Ist das der Fall, erfolgt ein Downsampling, bei dem nur jeder n-te Pixel berücksichtigt wird. Danach werden die Koordinaten um ihren Mittelpunkt zentriert und in 3D-Koordinaten (x,y,z) transformiert, wobei auch die Höhe angepasst wird. Abschließend wird die finale Liste der 3D-Koordinaten zusammen mit der tatsächlichen Drohnenanzahl zurückgegeben, der Prozess ist damit abgeschlossen.

2 Analyse der Produktfunktionen

Dieses Kapitel analysiert die einzelnen Produktfunktionen aus dem Pflichtenheft, um die spätere Architekturentwicklung zu ermöglichen. Jede Funktion wird in einem eigenen Unterkapitel dokumentiert.

2.1 Analyse von Funktion F10: Konfiguration der Show

Diese Funktion ermöglicht dem Benutzer die Planung der Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen. Die Anforderungen RM1, RM2, RM4, RM6 und RM7 sind hierbei zentral.

Beim Anlegen einer neuen Szene klickt der Nutzer in der GUI auf "Neue Szene erstellen". Daraufhin wird im Hintergrund der ShowKonfigurator aufgerufen, etwa über eine Methode wie `neueSzeneAnlegen()`. Diese Funktion ruft die ShowKonfigurator-Komponente auf, welche mit Hilfe des Projektmanagement-Moduls ein neues leeres Szenen-Objekt erstellt. Anschließend gibt das Projektmanagement Modul eine Bestätigung. Der ShowKonfigurator informiert daraufhin die GUI, welche die Anzeige aktualisiert und dem Nutzer die neu erstellte Szene und weitere Eingabefelder zeigt, um die Szene nun zu konfigurieren.

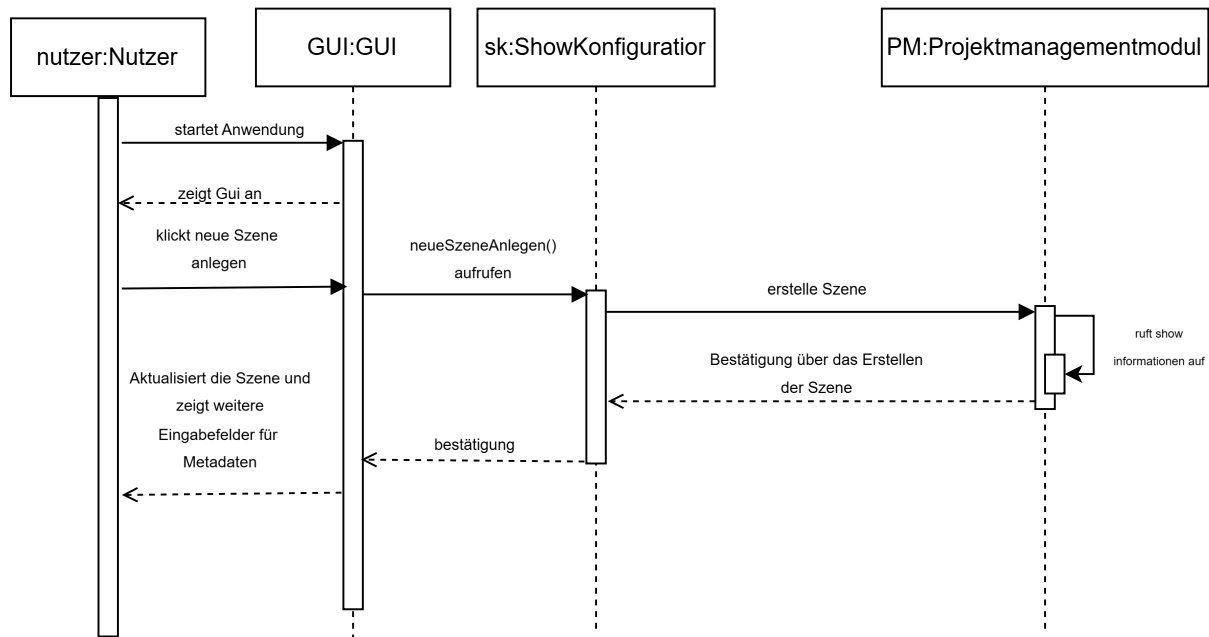


Abbildung 2.1: Sequenzdiagramm für F10

2.2 Analyse von Funktion F11: Szene anlegen und Metadaten erfassen

Beim Anlegen einer neuen Szene wird der Nutzer in der GUI aufgefordert, grundlegende Metadaten wie Name, Dauer und Drohnenanzahl einzugeben. Diese Metadaten werden anschließend vom GUI-Element an den ShowKonfigurator übergeben, der sie nutzt, um eine neue Szene zu initialisieren. Der ShowKonfigurator reicht die Metadaten weiter an das Projektmanagement-Modul, welches ein neues Szenenobjekt erstellt und die übergebenen Informationen speichert. Nach erfolgreicher Anlage gibt das Projektmanagement Modul eine Bestätigung oder das erstellte Objekt zurück, woraufhin die GUI entsprechend aktualisiert wird und dem Nutzer die leere Szene oder weitere Bearbeitungsmöglichkeiten anzeigt.

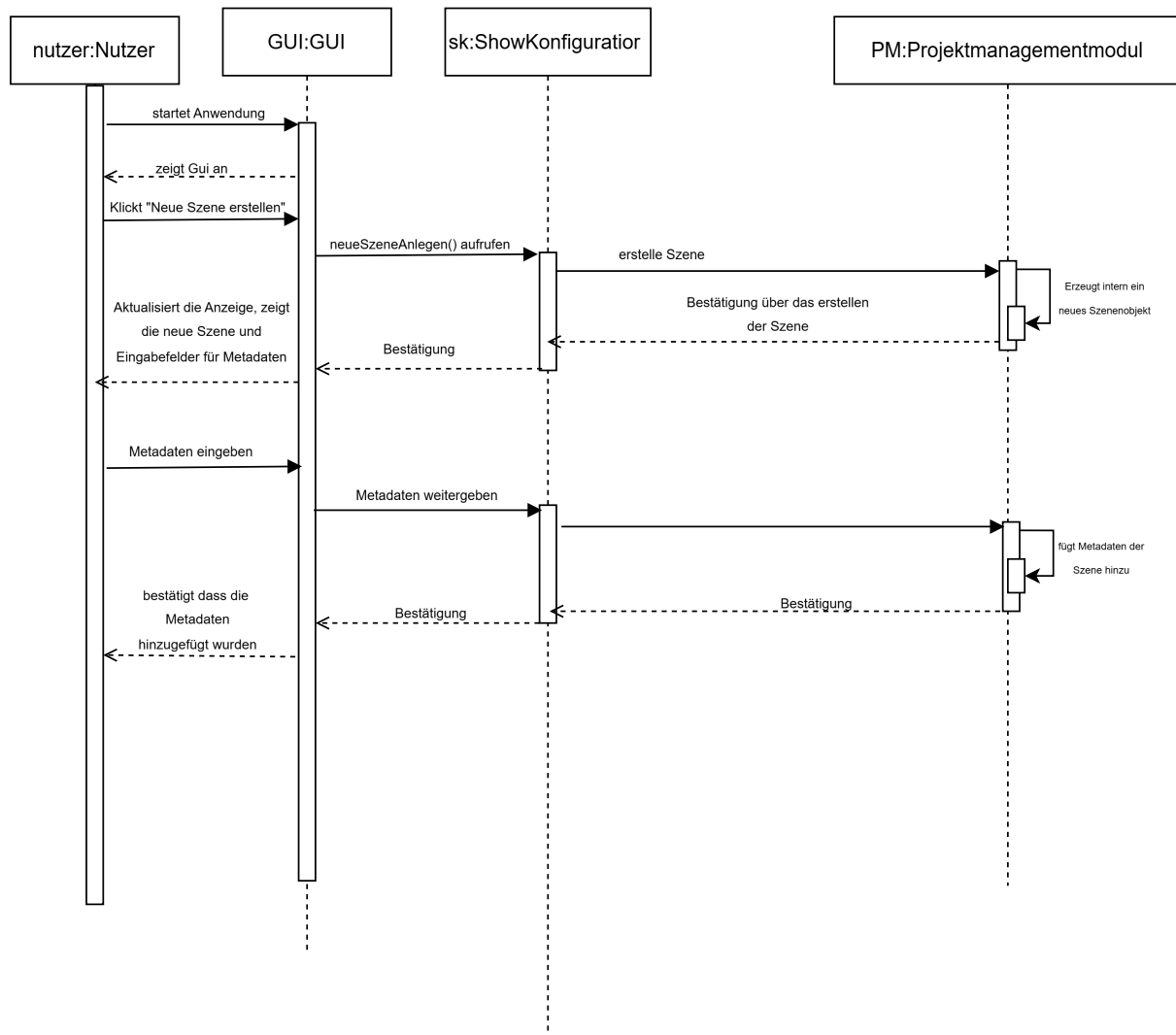


Abbildung 2.2: Sequenzdiagramm für F11

2.3 Analyse von Funktion F12: Formation und Bewegung definieren

Beim Auswählen einer Formation ist eine Szene bereits in der GUI geöffnet und aktiv, die SzenenVerwaltung kennt die aktuelle Drohnenanzahl. Der Nutzer klickt auf „Formation auswählen“, woraufhin die GUI dem ShowKonfigurator die aktuelle Szene übermittelt. Der ShowKonfigurator fragt in der FormationsBibliothek nach verfügbaren Formationen und erhält eine Liste.. Diese Optionen werden der GUI angezeigt. Nachdem der Nutzer eine Formation ausgewählt hat, teilt die GUI dem ShowKonfigurator die gewählte Formation mit. Der ShowKonfigurator

holt über die SzenenVerwaltung die Szenendetails. Der ShowKonfigurator wendet die Formation an, indem er das Projektmanagement-Modul die Anwendung der Formation übergibt. Das Projektmanagement-Modul berechnet basierend auf Formation und Drohnenanzahl die exakten Positionen jeder Drohne und aktualisiert deren Zustände. Nach Bestätigung der Anwendung aktualisiert der ShowKonfigurator die GUI.

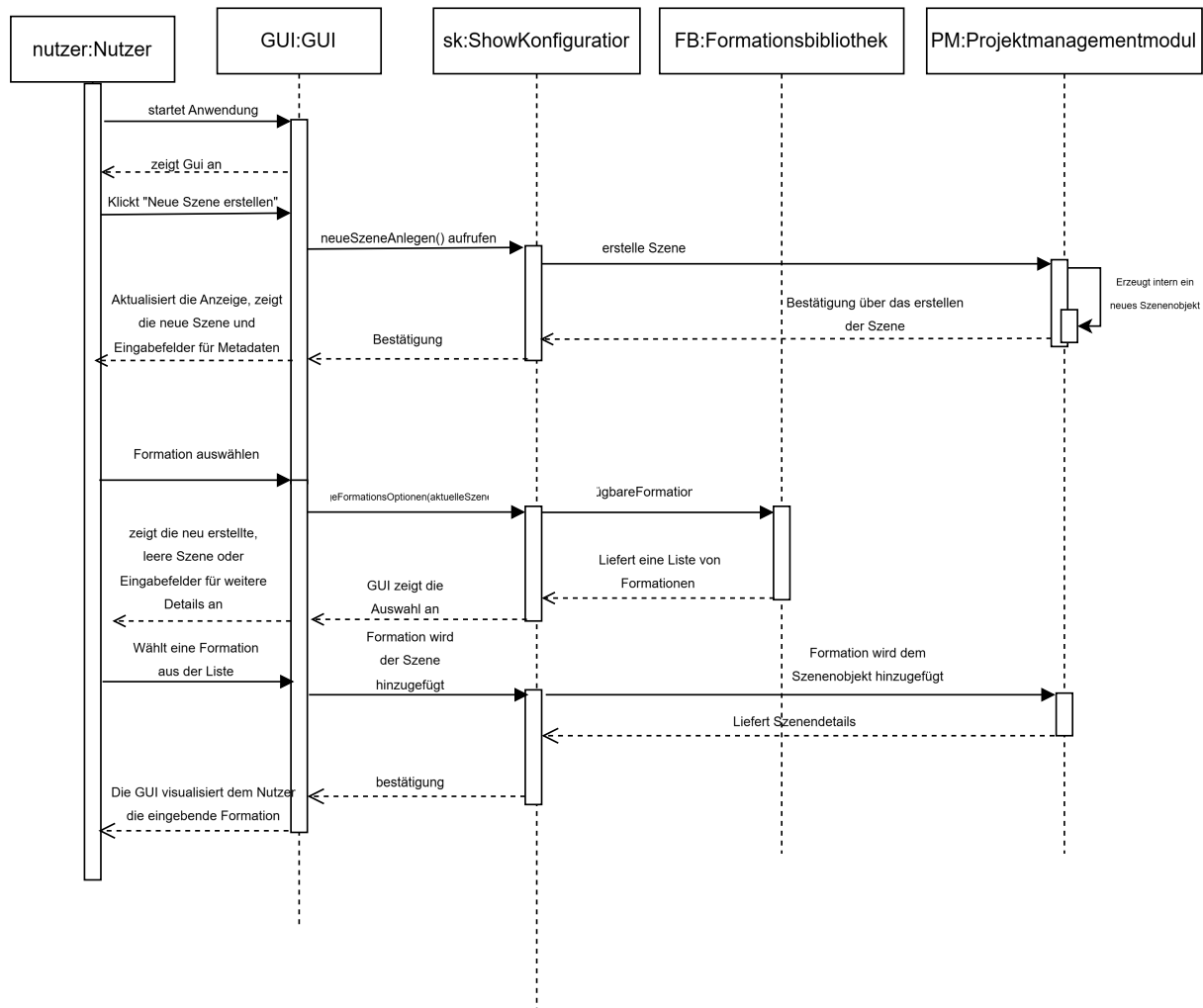


Abbildung 2.3: Sequenzdiagramm für F12

2.4 Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren

Beim Konfigurieren eines Lichteffekts ist zunächst eine Szene in der GUI geöffnet und enthält bereits Drohnen. Der Nutzer klickt auf „Lichteffekt hinzufügen“ oder wählt in der Timeline bzw. die gewünschten Drohnen aus. Die GUI übermittelt daraufhin an den ShowKonfigurator die

aktuelle Szene, die ausgewählten Drohnen und den Zeitpunkt in der Timeline. Der ShowKonfigurator holt über die LichtEffektBibliothek eine Liste verfügbarer Effekte und zeigt diese der GUI an. Nachdem der Nutzer einen Effekt (beispielsweise „Rot Dauerlicht“) samt optionaler Parameter wie Dauer und Helligkeit ausgewählt hat, übergibt die GUI die Daten zurück an den ShowKonfigurator. Anschließend aktualisiert der ShowKonfigurator die GUI, sodass die Timeline und gegebenenfalls eine Vorschau den neu angelegten Lichteffect anzeigen.

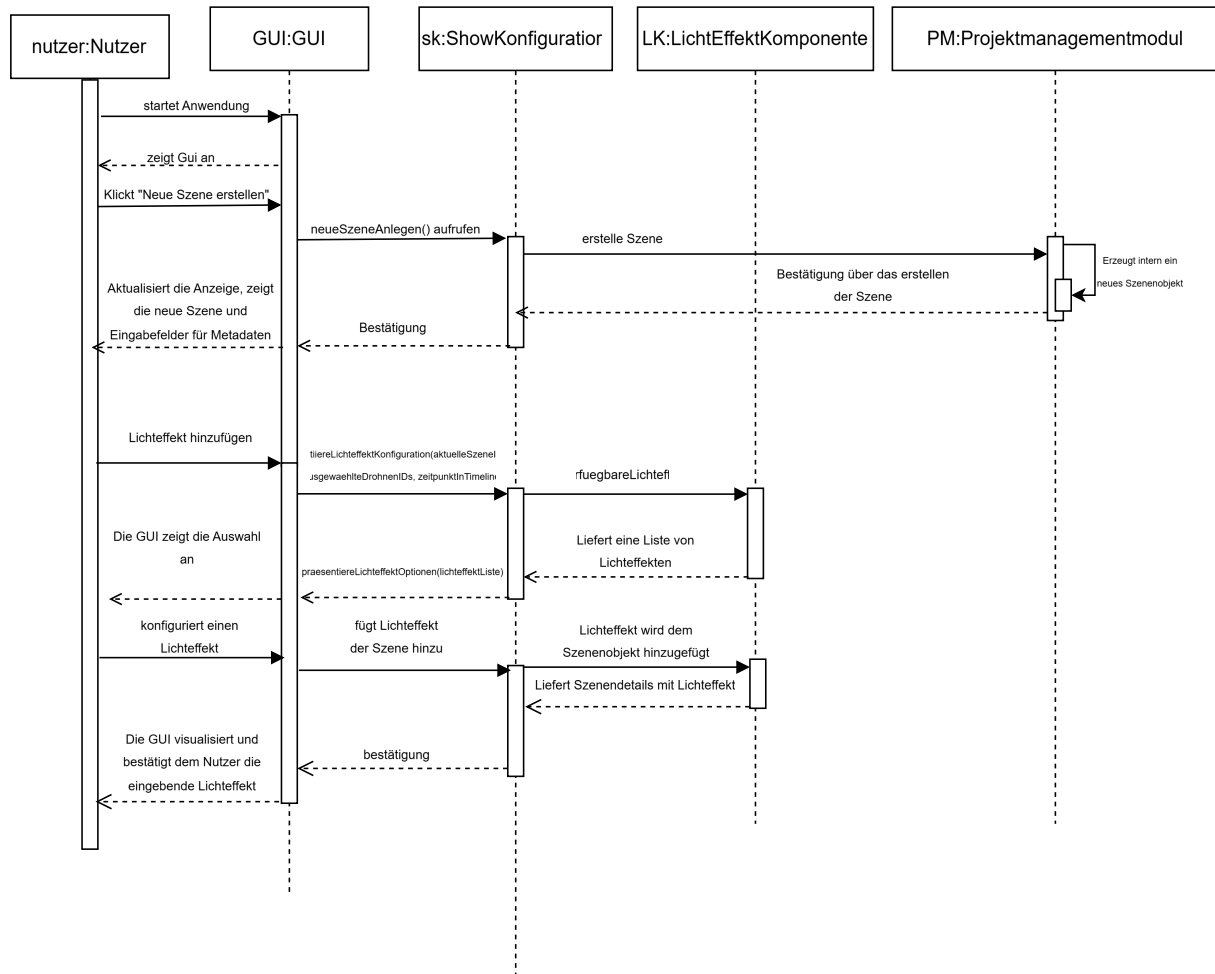


Abbildung 2.4: Sequenzdiagramm für F13

2.5 Analyse von Funktion F14: Szene speichern und zur Simulation übergeben

Die Simulation einer Szene beginnt, wenn der Nutzer in der geöffneten und vollständig eingerichteten Szene auf „Simulation starten“ klickt. Die GUI leitet diesen Befehl an den ShowKonfi-

gurator weiter, der daraufhin alle benötigten Szenendaten von dem Projektmanagement-Modul anfordert. Nachdem der ShowKonfigurator die vollständigen Informationen zurückerhalten hat, kann er die Szene, falls gewünscht, zunächst über die SpeicherKomponente sichern. Anschließend bereitet der ShowKonfigurator die Daten in einem simulationsgerechten Format auf und übergibt sie an die Simulationsbibliothek (Pybullet). Sobald die Simulationsbibliothek den Eingang der Szene bestätigt und die Simulation initialisiert hat, informiert der ShowKonfigurator die GUI darüber, dass die Simulation gestartet wurde. Die GUI wechselt daraufhin in den Simulationsmodus und zeigt dem Nutzer an, wie die Drohneninformationen, Bewegungsabläufe und Lichteffekte ablaufen.

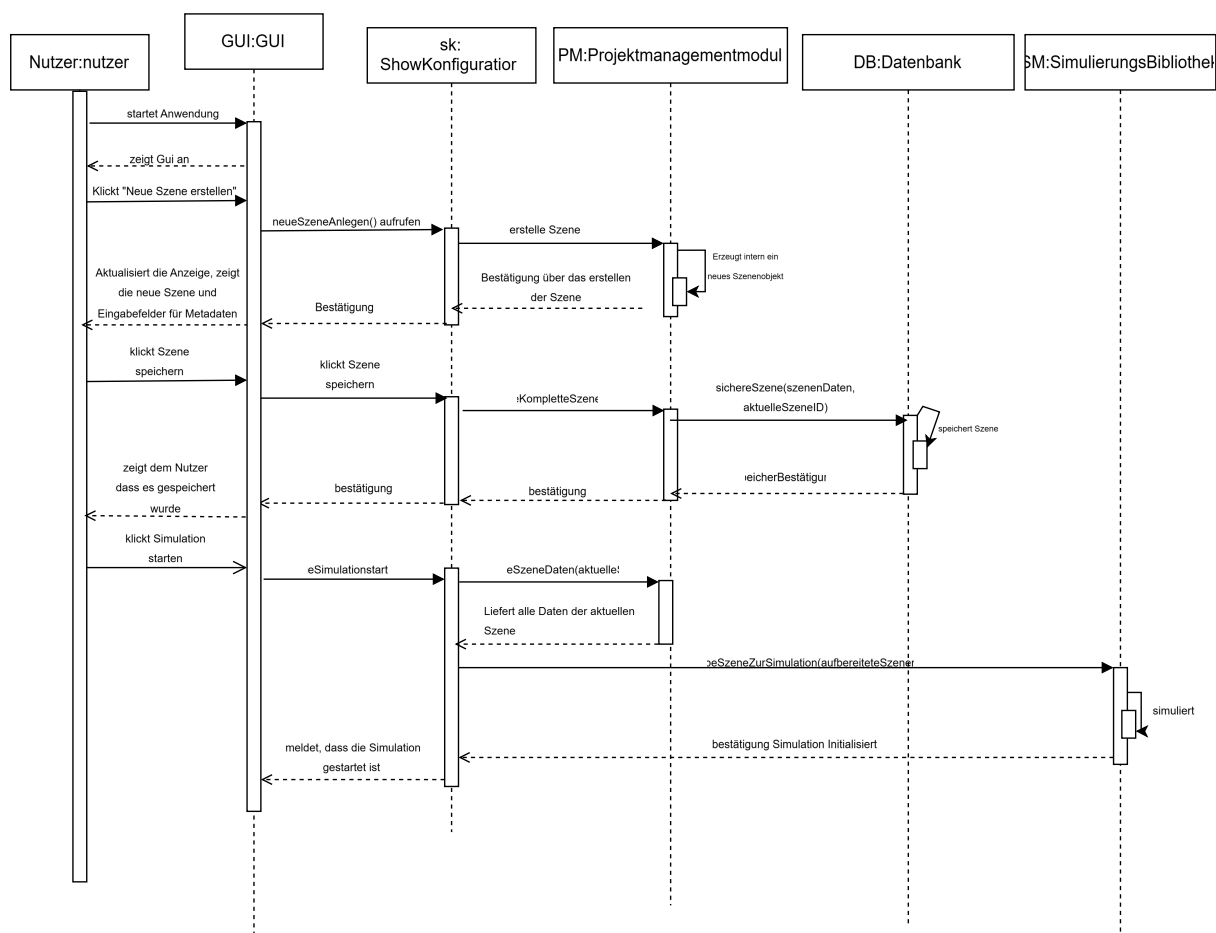


Abbildung 2.5: Sequenzdiagramm für F14

2.6 Analyse von Funktion F20: Choreographie simulieren

Beim Starten der Simulation lädt der Nutzer über die GUI-Modul die gewünschte Show (entweder die aktuell geladene oder eine gespeicherte) zur Anzeige und klickt auf „Show simulieren“.

Die GUI reicht den Befehl an das Konfigurationsmodul weiter, das alle relevanten Show-Daten vom Projektmanagement-Modul abrufen. Anschließend übergibt das Konfigurationsmodul diese vollständigen Daten an die Simulationsbibliothek. Dort wird die 3D-Umgebung initialisiert und die Drohnenobjekte entsprechend der ersten Szene positioniert. Sobald das Simulationsmodul die Daten erfolgreich übernommen hat, läuft die Simulation ab. Nachdem es die Simulation beendet hat, meldet es dem Konfigurationsmodul, dass die Show zu Ende ist. Dieses Modul leitet wiederum die Bestätigung an das Gui weiter. Das Gui wird wieder aktiv und der Nutzer kann erneut über das Programm die Show konfigurieren oder erneut simulieren.

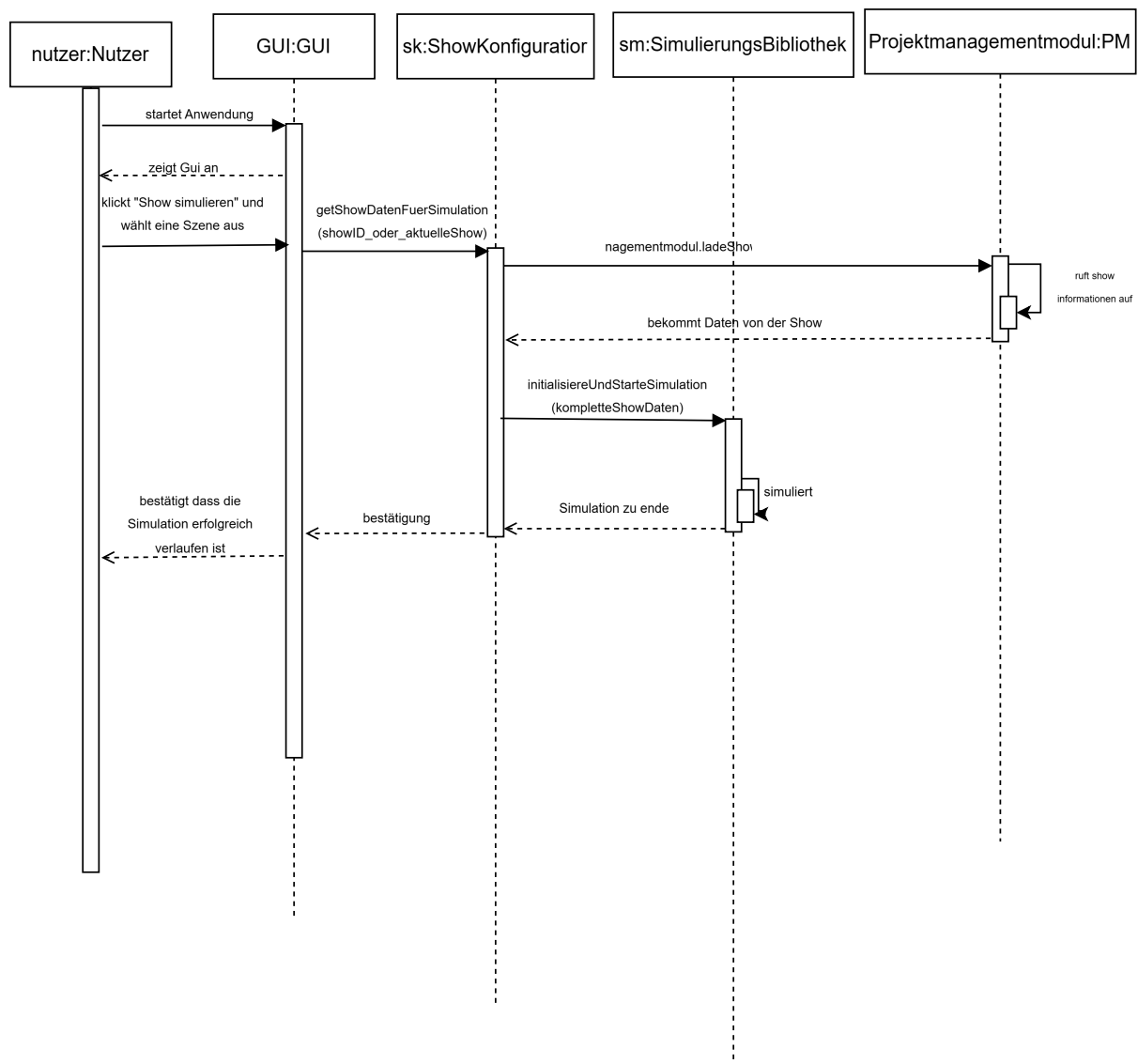


Abbildung 2.6: Sequenzdiagramm für F20

2.7 Analyse von Funktion F30: Projekt verwalten

Die Funktion F30: Projekt verwalten beschreibt das Speichern, Laden und Exportieren von Projekten innerhalb des SkyOrchestrator-Systems. Sobald der Benutzer in der grafischen Oberfläche eine entsprechende Aktion auslöst, beispielsweise über „Projekt speichern“ oder „Projekt laden“, leitet die GUI den Befehl an den ProjektManager weiter. Dieser übernimmt die Verarbeitung des Befehls, etwa durch Serialisierung der Projektdaten und Weitergabe an das Dateisystem im Falle eines Speichervorgangs oder durch Einlesen und Validierung bestehender Daten beim Laden eines Projekts. Der ProjektManager kommuniziert dabei direkt mit dem Dateisystem, ruft die jeweiligen Methoden auf und gibt nach erfolgreicher oder fehlerhafter Ausführung eine Rückmeldung an die GUI zurück. Diese aktualisiert daraufhin die Benutzeroberfläche und zeigt dem Benutzer das Ergebnis an. Auch Exportvorgänge werden über dieses Modul abgewickelt, indem Projektinformationen in ein geeignetes Austauschformat überführt und gespeichert werden. Die folgende Abbildung zeigt die beteiligten Komponenten sowie die Austauschbeziehungen im Rahmen eines typischen Speichervorgangs.

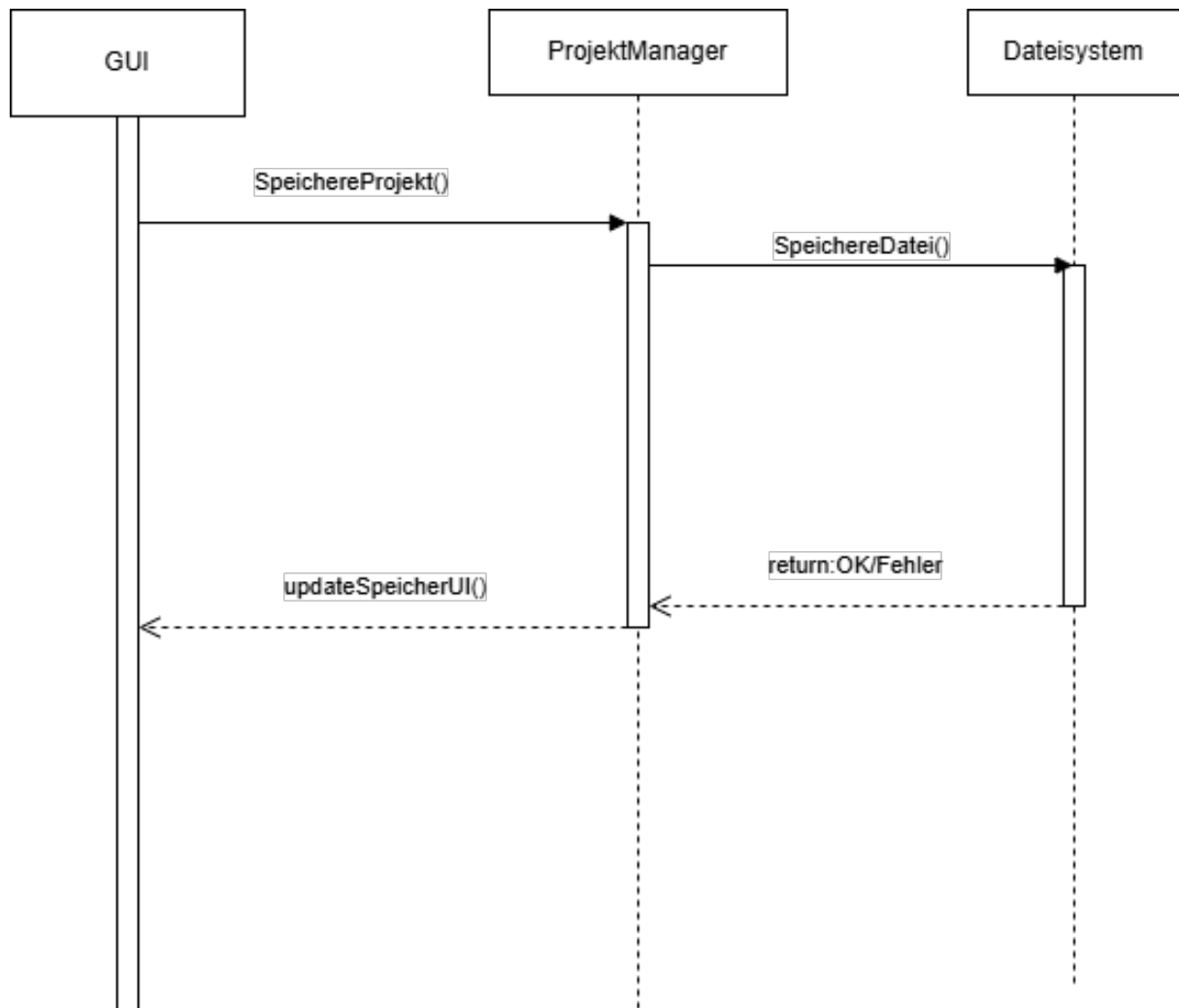


Abbildung 2.7: Sequenzdiagramm für F30

2.8 Analyse von Funktion F40: Szenenprüfung auf Eingabefehler

Die Funktion F40: Szenenprüfung auf Eingabefehler ermöglicht die automatisierte Überprüfung einer konfigurierten Szene auf Vollständigkeit und Plausibilität der eingegebenen Daten. Sobald der Benutzer über die grafische Benutzeroberfläche eine entsprechende Aktion auslöst, etwa durch Auswahl des Befehls „Szene prüfen“ oder durch den Versuch, eine Szene zu simulieren oder zu speichern, übermittelt die GUI eine Prüfanforderung an die Validierungskomponente. Diese greift auf die aktuellen Szenendaten der SzenenVerwaltung zu, analysiert sämtliche Parameter und Eingaben der Szene, und überprüft dabei insbesondere Pflichtfelder, Wertebereiche, zeitliche Abfolgen sowie geometrische Kollisionen oder doppelte Zuordnungen. Im Falle fehler-

hafter oder unvollständiger Daten generiert die Validierung eine strukturierte Fehlerliste und sendet diese an die GUI zurück. Die grafische Oberfläche visualisiert die Rückmeldung in geeigneter Form, beispielsweise durch Fehlermarkierungen oder Meldungsfenster, und ermöglicht dem Benutzer die gezielte Korrektur der betroffenen Elemente. Die folgende Abbildung zeigt den Nachrichtenaustausch zwischen den beteiligten Komponenten beim Ablauf einer Szenenprüfung.

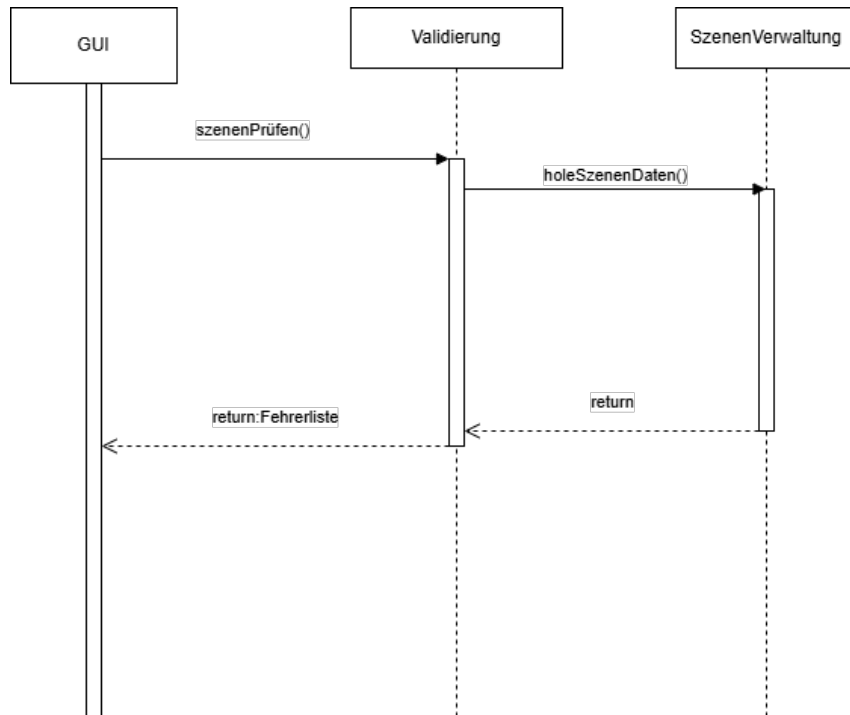


Abbildung 2.8: Sequenzdiagramm für F40

3 Resultierende Softwarearchitektur

Dieses Kapitel beschreibt, basierend auf den analysierten Produktfunktionen aus Kapitel 2, die Softwarearchitektur von SkyOrchestrator. Die Architektur gliedert sich in mehrere Komponenten und Subsysteme, die jeweils klar definierte Aufgaben übernehmen.

3.1 Komponentenspezifikation

Die Komponenten des Systems sind im Komponentendiagramm (Abbildung 3.1) dargestellt.

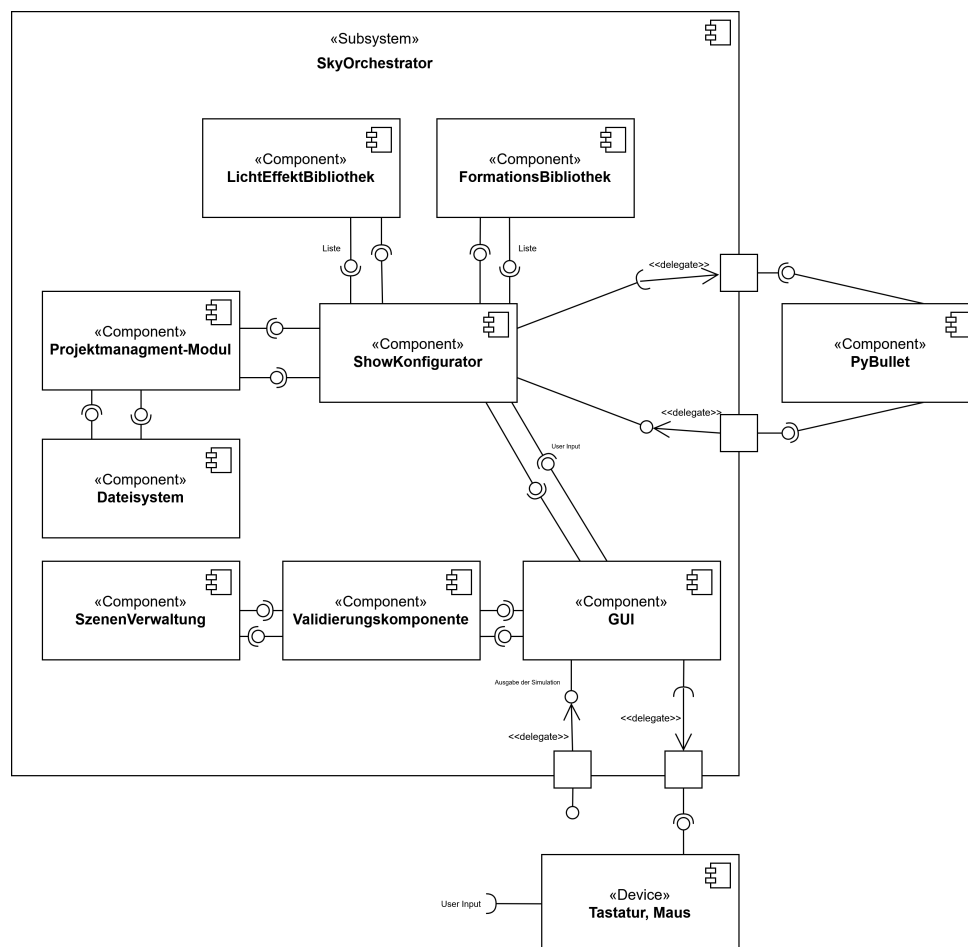


Abbildung 3.1: Komponentendiagramm

Folgend werden die Funktionen dieser Komponenten beschrieben.

Komponente $\langle C10 \rangle$: $\langle$ Graphical User Interface (GUI) $\rangle$

Die GUI ist primär für die Interaktion mit dem Nutzer und die Visualisierung des Systemzustands verantwortlich. Sie stellt die gesamte Benutzeroberfläche für die Konfiguration von Szene und Show bereit. Ihre Hauptaufgabe umfassen die Entgegennahme und Verarbeitung von Nutzereingaben, um eine intuitive Bedienung zu gewährleisten. Des Weiteren ist die GUI für die Anzeigen von Rückmeldungen, Fehler und Simulationsergebnissen zuständig, wodurch der Nutzer stets über den aktuellen Systemstatus informiert wird. Die Kommunikation mit dem ShowKonfigurator erfolgt über definierte Schnittstellen, um die Steuerung der Anwendung zu ermöglichen und die Interaktion mit den weiteren Komponenten des Systems zu orchestrieren.

Komponente $\langle C20 \rangle$: $\langle$ ShowKonfigurator $\rangle$

Der ShowKonfigurator dient als zentrale Steuerungs- und Koordinationskomponente zwischen der GUI, dem Projektmanagement-Modul, der Formationsbibliothek, der Lichteffektbibliothek und der Simulationsbibliothek. Er verarbeitet die Nutzereingaben, verwaltet die aktuelle Szene sowie die zugehörigen Show-Daten und sorgt für die Anwendung der ausgewählten Formationen und Lichteffekte. Zudem übernimmt der ShowKonfigurator die Übergabe der konfigurierten Daten an das Projektmanagement-Modul zur weiteren Verarbeitung und stellt sicher, dass alle beteiligten Komponenten synchron zusammenarbeiten.

Komponente $\langle C30 \rangle$: $\langle$ Projektmanagement-Modul $\rangle$

Das Projektmanagement-Modul übernimmt die zentrale Verwaltung und Organisation der Projektdaten innerhalb des SkyOrchestrator-Systems. Es ist verantwortlich für die Berechnung der Drohnenpositionen basierend auf den ausgewählten Formationen und Bewegungsabläufen. Darüber hinaus sorgt das Modul für das Speichern und Laden von Projekten, um eine verlässliche Verwaltung der Show-Konfigurationen zu gewährleisten. Für die Simulation bereitet das Projektmanagement-Modul die Szenendaten auf und übergibt diese an die Simulationsbibliothek.

Komponente $\langle C40 \rangle$: $\langle$ FormationsBibliothek $\rangle$

Die FormationsBibliothek stellt eine Sammlung vordefinierter Drohnenformationen bereit, die vom ShowKonfigurator zur Konfiguration der Shows genutzt werden. Sie verwaltet die verfügbaren Formationstypen und ermöglicht deren Erweiterung, um auf unterschiedliche Anforderungen reagieren zu können.

Komponente $\langle C50 \rangle$: $\langle$ LichtEffektBibliothek $\rangle$

Die LichtEffektBibliothek stellt eine Sammlung vordefinierter Lichteffekte sowie deren zugehöriger Parameter bereit, die vom ShowKonfigurator zur Konfiguration der Shows verwendet werden. Sie verwaltet die verfügbaren Effekte und ermöglicht deren Erweiterung, um eine vielfältige Gestaltung von Lichtshows zu ermöglichen.

Komponente <C60>: <Simulationsbibliothek (PyBullet)>

Die Simulationsbibliothek PyBullet ist verantwortlich für die physikalische Simulation der Drohnenchoreographie in einer dreidimensionalen Umgebung. Sie initialisiert die Szene basierend auf den übergebenen Daten und steuert den Ablauf der Simulation. Die Bibliothek gibt während des Simulationsprozesses kontinuierlich Rückmeldungen zum Simulationsstatus an den ShowKonfigurator.

Komponente <C70>: <Validierungskomponente>

Die Validierungskomponente überprüft die Szenendaten auf Vollständigkeit und Plausibilität. Bei fehlerhaften oder unvollständigen Eingaben generiert sie Fehlerlisten, die die gefundenen Probleme beschreiben. Diese Fehler werden an die GUI übermittelt und über diese dargestellt, was dem Nutzer ermöglicht, gezielt Korrektur an den fehlerhaften Elementen vorzunehmen.

Komponente <C80>: <DatenSysteme>

Die Komponente DatenSysteme ist für die zentrale Verwaltung und Organisation aller persistenter Daten im System zuständig. Sie stellt Schnittstellen zum Speichern, Laden und Exportieren von Projektdaten bereit und sorgt für die Datenintegrität sowie die Konsistenz der gespeicherten Informationen.

Komponente <C90>: <SzenenVerwaltung>

Die SzenenVerwaltung ist verantwortlich für die Organisation und Verwaltung aller Szenenobjekte innerhalb des Systems. Sie verwaltet die Metadaten, Formationen, Bewegungen und Lichteffekte der Szenen und stellt diese Daten anderen Komponenten für Konfiguration, Simulation und Validierung bereit.

3.2 Schnittstellenspezifikation

In diesem Kapitel werden die einzelnen Schnittstellen jeder Komponente dargestellt.

Schnittstelle <I10>: <GUI $\leftrightarrow$ ShowKonfigurator>

Operation	Beschreibung
<createNewScene()>	Erstellt eine neue Szene zum Konfigurieren der Show
<sendFormation(formationID)>	Übermittelt die vom Nutzer gewählte Formation
<sendLightEffect(effectID, parameters)>	Übermittelt die vom Nutzer ausgewählten Lichteffekte und Parameter
<runSimulation()>	Signalisiert den Start der Simulation

Schnittstelle $\langle I10 \rangle$: $\langle \text{GUI} \longleftrightarrow \text{Validierungskomponente} \rangle$

$\langle \text{validateScene}(\text{sceneData}) \rangle$	Prüft die Szene auf Eingabefehler und gibt Fehlerliste zurück
--	---

Schnittstelle $\langle I20 \rangle$: $\langle \text{ShowKonfigurator} \longleftrightarrow \text{Projektmanagement-Modul} \rangle$

Operation	Beschreibung
$\langle \text{applyFormation}(\text{Formation}, \text{Drone-Count}) \rangle$	Berechnet und aktualisiert Drohnenpositionen basierend auf Formation
$\langle \text{getSceneDetails}(\text{SceneID}) \rangle$	Liefert Details zur aktuellen Szene
$\langle \text{saveProject}(\text{ProjectData}) \rangle$	Speichert Projektdaten
$\langle \text{loadProject}(\text{ProjectID}) \rangle$	Lädt Projektdaten
$\langle \text{exportProject}(\text{Format}) \rangle$	Exportiert Projektdaten in ein Austauschformat

Schnittstelle $\langle I10 \rangle$: $\langle \text{ShowKonfigurator} \longleftrightarrow \text{FormationsBibliothek} \rangle$

$\langle \text{listAvailableFormations}() \rangle$	Liefert eine Liste verfügbarer Formationen
--	--

Schnittstelle $\langle I10 \rangle$: $\langle \text{ShowKonfigurator} \longleftrightarrow \text{LichtEffektBibliothek} \rangle$

$\langle \text{listAvailableLightEffects}() \rangle$	Liefert eine Liste verfügbarer Lichteffekte
--	---

Schnittstelle $\langle I10 \rangle$: $\langle \text{ShowKonfigurator} \longleftrightarrow \text{Simulationsbibliothek} \rangle$

$\langle \text{initSimulation}(\text{SceneData}) \rangle$	Initialisiert die Simulation mit den Szenendaten
$\langle \text{startSimulation}() \rangle$	Startet die Simulation
$\langle \text{stopSimulation}() \rangle$	Beendet die Simulation
$\langle \text{getSimulationStatus}() \rangle$	Liefert den aktuellen Status der Simulation

Nach der Initialisierung der GUI stehen dem Benutzer drei Hauptmodi zur Verfügung: **Texteingabemodus**, **Bildmodus** und **Formationsmodus**. Die Auswahl des Modus erfolgt anhand der Benutzeraktion.

- **Texteingabemodus:**

Im Texteingabemodus wird das Texteingabefeld aktiviert, während die Upload-Buttons deaktiviert sind. Nach der Eingabe wird der Text auf Gültigkeit überprüft. Bei gültigem Text können ggf. Animationen definiert werden, andernfalls erfolgt eine entsprechende Rückmeldung.

- **Bildmodus:**

Im Bildmodus sind die Upload-Buttons aktiviert und das Texteingabefeld deaktiviert. Der Benutzer kann ein Bild hochladen, das anschließend als Vorschau angezeigt wird. Bei erfolgreichem Upload erfolgt die Weiterleitung zum nächsten Schritt. Bei Fehlern wird eine Fehlermeldung angezeigt.

- **Formationsmodus:**

Im Formationsmodus wählt der Benutzer eine vordefinierte Formation aus. Das Texteingabefeld und die Upload-Buttons sind deaktiviert. Nach Auswahl wird die Formation geladen und eine Bildvorschau angezeigt.

- **Drawmodus:**

Im Drawmodus zeichnet der Benutzer und kann diese Zeichnung in eine Formation umwandeln lassen. Das Texteingabefeld und die Upload-Buttons sind deaktiviert. Nach Auswahl eine Zeichenumgebung geladen und nach dem der Nutzer mit der Zeichnung fertig ist, kann er diese wieder beenden.

Nach Abschluss der Eingabe in einem der drei Modi erfolgt die **Parameterkonfiguration**. Hier werden Hintergrundoptionen, Drohnenmodelle und Lichtkonfigurationen geladen und ausgewählt. Anschließend wird eine Validierung durchgeführt. Nach erfolgreicher Validierung kann die Simulation gestartet werden, woraufhin die GUI geschlossen wird.

Bedeutung für die Wiederverwendung

Das beschriebene Protokoll stellt sicher, dass die Komponenten in einer konsistenten Reihenfolge und mit den jeweils gültigen Eingaben verwendet werden. Durch die klare Trennung der Modi und die explizite Steuerung der Zustandsübergänge wird eine fehlerfreie und wiederverwendbare Nutzung der Komponenten ermöglicht. Dies ist insbesondere für zukünftige Erweiterungen oder die Integration in andere Projekte von Vorteil.

3.3.2 ShowKonfigurator

Die Abläufe und Zustände der Komponente ShowKonfigurator werden im Folgenden anhand des Zustandsdiagramms erläutert.

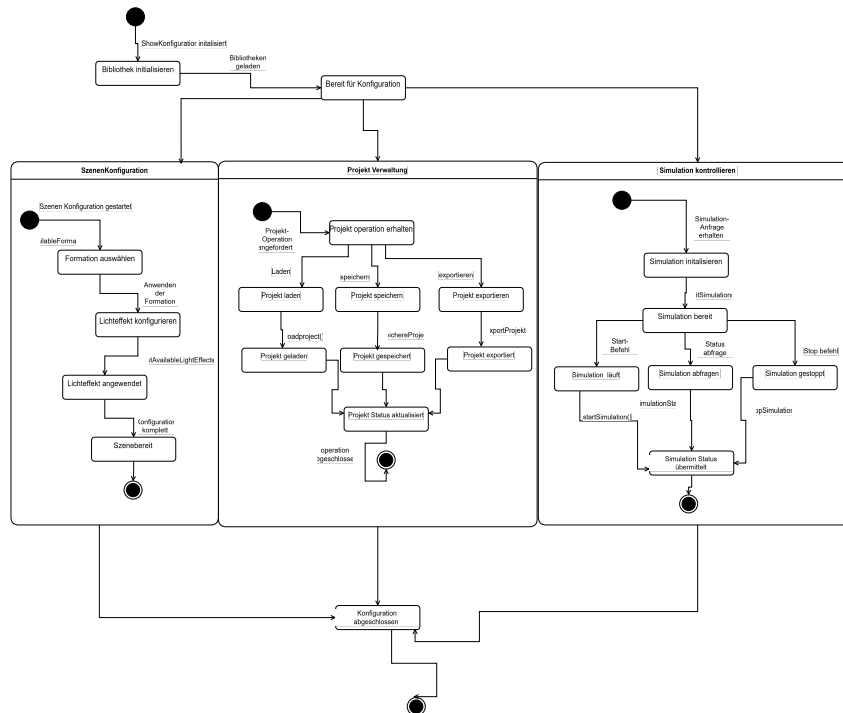


Abbildung 3.3: Zustandsdiagramm: ShowKonfigurator

Nach der Initialisierung des ShowKonfigurators und dem erfolgreichen Laden der benötigten Bibliotheken befindet sich das System im Zustand **Bereit für Konfiguration**. Von hier aus kann der Benutzer zwischen drei Hauptbereichen wählen: **Szenenkonfiguration**, **Projektverwaltung** und **Simulation kontrollieren**.

- **Szenenkonfiguration:**

In diesem Bereich kann der Benutzer eine Formation auswählen und anschließend Lichteffekte konfigurieren. Nach Anwenden der gewählten Effekte ist die Konfiguration abgeschlossen und die Szene ist bereit für weitere Schritte.

- **Projektverwaltung:**

Hier stehen Funktionen zum Laden, Speichern und Exportieren von Projekten zur Verfügung. Nach der Auswahl einer Projektoperation (z. B. Laden oder Speichern) wird der entsprechende Vorgang ausgeführt und der Projektstatus aktualisiert. Nach Abschluss der Operation kehrt das System in den Ausgangszustand zurück.

- **Simulation kontrollieren:**

In diesem Bereich kann eine Simulation initialisiert, gestartet, gestoppt oder ihr Status abgefragt werden. Nach dem Start oder Stopp der Simulation bzw. nach einer Statusabfrage wird der aktuelle Status an das System übermittelt.

Sobald alle gewünschten Konfigurationen und Operationen abgeschlossen sind, wird der Zustand **Konfiguration abgeschlossen** erreicht. Das Zustandsdiagramm visualisiert die möglichen Abläufe und Übergänge zwischen den einzelnen Zuständen und dient als Grundlage für die Implementierung sowie die spätere Wartung der Komponente.

Bedeutung für die Wiederverwendung

Das Zustandsdiagramm des ShowKonfigurators sorgt für eine klare Struktur und definiert die Abläufe der einzelnen Funktionsbereiche eindeutig. Dadurch wird die Komponente leicht verständlich, gut wartbar und kann problemlos in anderen Projekten wiederverwendet oder erweitert werden. Die modulare Organisation erleichtert Anpassungen und fördert die nachhaltige Nutzung in verschiedenen Anwendungskontexten.

3.3.3 Projektmanagement-Modul

Die Abläufe und Zustände der Komponente Projektmanagement werden im Folgenden anhand des Zustandsdiagramms erläutert.

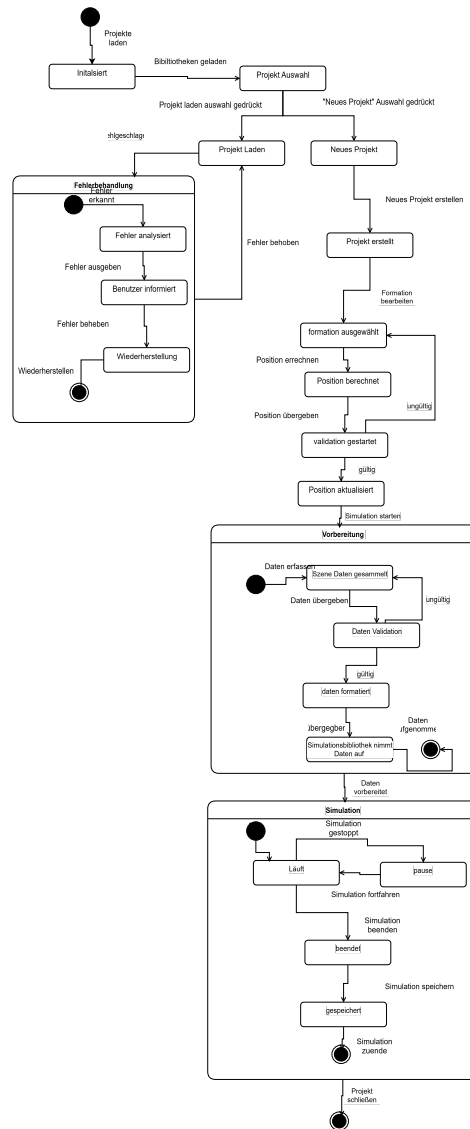


Abbildung 3.4: Zustandsdiagramm: Projektmanagement-Modul

Nach der Initialisierung der Komponente gelangt das System in den Zustand **Projekt Auswahl**. Hier kann der Benutzer entweder ein bestehendes Projekt laden oder ein neues Projekt anlegen.

- **Projekt Laden:** Wird ein Projekt geladen, prüft das System auf mögliche Fehler. Im Fehlerfall erfolgt eine Übergabe an die Fehlerbehandlung, in der der Fehler analysiert, der

Benutzer informiert und eine Wiederherstellung versucht wird. Nach erfolgreicher Wiederherstellung wird der Ladevorgang erneut gestartet.

- **Neues Projekt:**

Nach dem Anlegen eines neuen Projekts befindet sich das System im Zustand **Bereit**. Der Benutzer kann nun eine Formation auswählen und deren Position bearbeiten. Nach Auswahl und Bearbeitung der Formation wird eine Validierung durchgeführt. Ist die Position gültig, wird sie aktualisiert und die Simulation kann gestartet werden. Bei ungültigen Daten erfolgt eine Rückmeldung an den Benutzer.

Wird die Simulation gestartet, beginnt die Phase der **Vorbereitung**. Hier werden zunächst alle szenenrelevanten Daten gesammelt und validiert. Nach erfolgreicher Validierung werden die Daten formatiert und an die Simulationsbibliothek übergeben. Im Fehlerfall wird der Benutzer informiert und die Vorbereitung kann erneut gestartet werden.

Im Zustand **Simulation** kann die Simulation entweder laufen oder pausiert werden. Nach Abschluss der Simulation werden die Ergebnisse gespeichert. Abschließend kann das Projekt geschlossen werden, womit der Ablauf endet.

Bedeutung für die Wiederverwendung

Das Zustandsdiagramm der Projektmanagement-Modul sorgt für eine klare Strukturierung aller Abläufe von der Projektauswahl über die Fehlerbehandlung bis zur Simulation. Die eindeutigen Zustandsübergänge erhöhen die Nachvollziehbarkeit und Wartbarkeit der Komponente. Durch die modulare Organisation lassen sich einzelne Bereiche gezielt anpassen oder erweitern, was die Wiederverwendung in anderen Projekten oder Anwendungskontexten erleichtert.

3.3.4 Simulationsbibliothek

Die Abläufe und Zustände der Simulationsbibliothek werden im Folgenden anhand des zugehörigen Zustandsdiagramms erläutert.

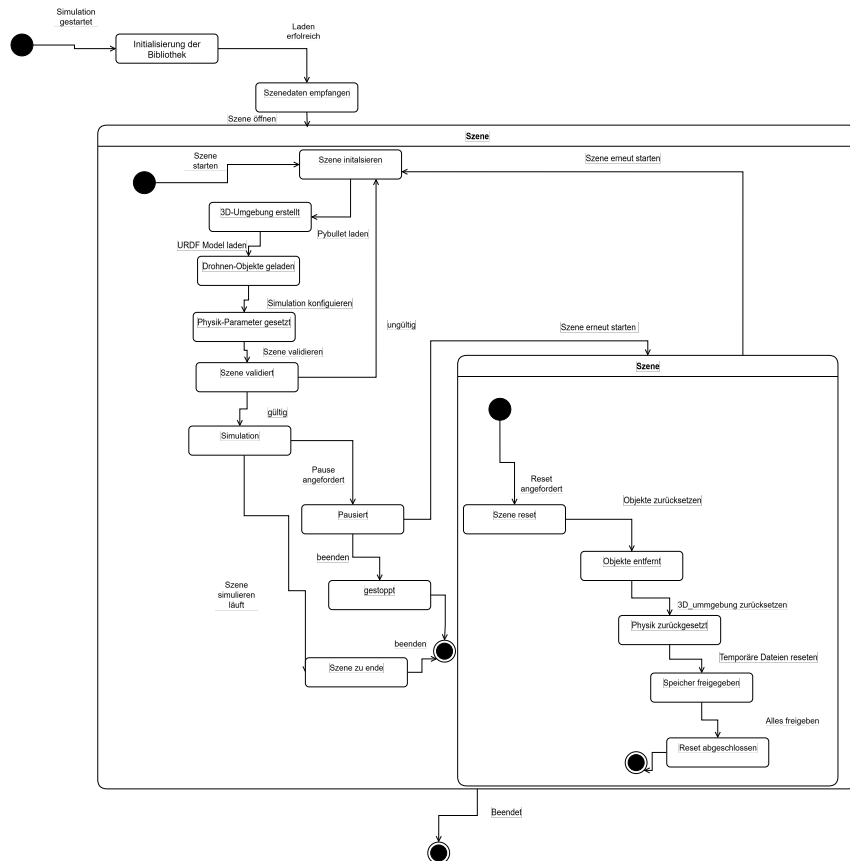


Abbildung 3.5: Zustandsdiagramm: Simulationsbibliothek

Nach der **Initialisierung der Bibliothek** und dem erfolgreichen Laden der erforderlichen Daten befindet sich das System im Zustand **Szene**. Hier werden die empfangenen Szenedaten verarbeitet und die Szene initialisiert.

Im Rahmen der Szeneninitialisierung wird zunächst die **3D-Umgebung erstellt**, anschließend werden die Drohnen-Objekte geladen und die physikalischen Parameter gesetzt. Nach Abschluss dieser Schritte erfolgt eine Validierung der Szene. Ist die Szene gültig, kann die Simulation gestartet werden. Bei ungültigen Szenendaten wird der Ablauf unterbrochen und es erfolgt ein Übergang in den Reset-Prozess.

Während der laufenden Simulation kann jederzeit eine Pause angefordert werden. Im Pausenmodus kann die Simulation gestoppt werden, wodurch die Szene zu Ende geführt wird. Nach Abschluss der Simulation ist der gesamte Ablauf beendet.

Wird ein **Reset der Szene** angefordert, gelangt das System in den Zustand **resetSzene**. Hier werden alle Objekte entfernt, die physikalischen Einstellungen zurückgesetzt und belegte Speicherressourcen freigegeben. Nach Abschluss des Resets kann die Szene erneut initialisiert werden, um einen neuen Simulationsdurchlauf zu ermöglichen.

Bedeutung für die Wiederverwendung

Das Zustandsdiagramm der Simulationsbibliothek sorgt für eine klare Trennung und Strukturierung aller Abläufe – von der Initialisierung über die Szenenverwaltung bis hin zur Simulation und zum Reset. Die explizite Abbildung aller Zustandsübergänge erleichtert die Wartung und Erweiterung der Bibliothek. Die modulare Organisation ermöglicht es, die Simulationsbibliothek flexibel in verschiedenen Projekten wiederzuverwenden und an unterschiedliche Anforderungen anzupassen.

4 Verteilungsentwurf

Da es sich bei unserem Projekt nicht um eine verteilte Anwendung im klassischen Sinne handelt, entfällt an dieser Stelle ein Verteilungsdiagramm.

Alle zentralen Komponenten wie GUI, Szenenverwaltung, ShowKonfigurator, Validierung und Simulation mittels PyBullet werden auf einem einzelnen System ausgeführt. Es findet keine verteilte Ausführung über mehrere physische oder logische Knoten statt. Die Anwendung wird lokal auf einem einzelnen Rechner ausgeführt. Alle Module laufen gemeinsam in einem einzigen Programm und müssen daher nicht über ein Netzwerk verteilt werden.

Ein Deployment-Diagramm zur Beschreibung der Verteilung wäre daher nicht sinnvoll und wird in dieser Dokumentation ausgelassen.

5 Implementierungsentwurf

In diesem Kapitel wird die technische Umsetzung der im Fachentwurf beschriebenen Systemarchitektur erläutert. Basierend auf der dort definierten Komponentenstruktur werden die zentralen Klassen und deren Beziehungen untereinander dargestellt. Ziel ist es, einen Überblick über die interne Struktur sowie die Interaktion der einzelnen Komponenten zu geben.

Für jede Hauptkomponente wird ein Klassendiagramm vorgestellt und beschrieben, wie deren Funktionalitäten durch Attribute und Methoden realisiert wurden. Dabei werden insbesondere die von außen zugänglichen Schnittstellen hervorgehoben, um die Zusammenarbeit der Module nachvollziehbar zu machen. Zusätzlich werden wichtige Abhängigkeiten zu externen Bibliotheken sowie genutzte Hilfsklassen aufgezeigt.

Die nachfolgenden Abschnitte widmen sich jeweils einer Komponente gemäß der Architekturübersicht aus Kapitel 3.

5.1 Implementierung von Komponente C10 – GUI

Die GUI ist die zentrale Schnittstelle zwischen dem Benutzer und dem SkyOrchestrator-System. Sie stellt Funktionen zur Verfügung, um Projekte zu erstellen, Szenen zu konfigurieren, Formationen festzulegen, Zeitachsen zu bearbeiten und Simulationen zu starten. Durch eine benutzerfreundliche Oberfläche ermöglicht sie die intuitive Steuerung komplexer Drohnenshows – auch für Nutzer:innen ohne technische Vorkenntnisse.

Die GUI kommuniziert über dedizierte Schnittstellen mit dem ShowKonfigurator ($\langle C20 \rangle$), um Benutzeraktionen in interne Systemprozesse zu übersetzen. Darüber hinaus interagiert sie mit dem Projektmanagement-Modul ($\langle C30 \rangle$) zur Verwaltung gespeicherter Szenen sowie mit der Simulationskomponente ($\langle C60 \rangle$), um die Simulation auszulösen und visuell darzustellen.

5.1.1 Paket-/Klassendiagramm

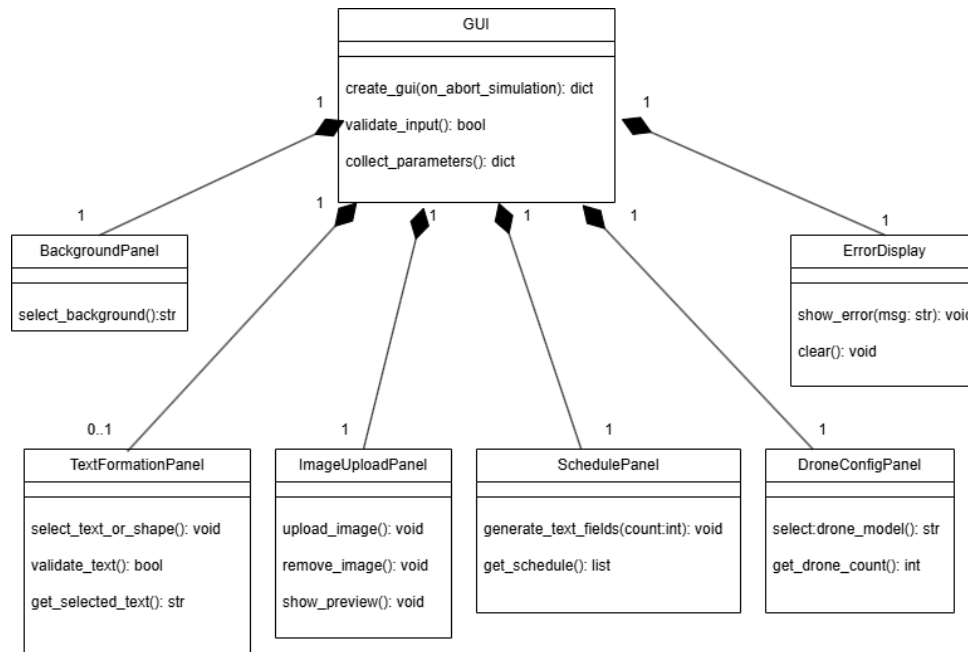


Abbildung 5.1: Klassendiagramm für Komponente <C10>

5.1.2 Erläuterung

Die GUI-Komponente ermöglicht die benutzerfreundliche Erstellung, Konfiguration und Steuerung von Drohnenshows. Sie stellt verschiedene Teilbereiche bereit, die jeweils für spezifische Eingaben und Konfigurationsaspekte verantwortlich sind. Der zentrale Einstiegspunkt ist die Klasse GUIController, die alle Subkomponenten orchestriert.

Beschreibung der GUI-Klassen

GUI<CL10>

Aufgabe:

Steuert den gesamten GUI-Aufbau, initialisiert und koordiniert die Unterkomponenten, validiert Eingaben und sammelt Nutzerdaten für die Simulation.

Attribute:

keine

Operationen:

- `create_gui(on_abort_simulation)`: Erzeugt alle GUI-Elemente und gibt ein Dictionary mit allen Nutzereingaben zurück.
- `validate_input()`: Prüft, ob alle erforderlichen Felder ausgefüllt und valide sind.
- `collect_parameters()`: Sammelt aktuelle Nutzereingaben aus den Unterkomponenten und bereitet sie für die Weiterverarbeitung vor.

Kommunikationspartner:

TextFormationPanel(⟨CL11⟩), BackgroundPanel(⟨CL12⟩), ImageUploadPanel(⟨CL13⟩), SchedulePanel(⟨CL14⟩), DroneConfigPanel(⟨CL15⟩), ErrorDisplay(⟨CL16⟩)

TextFormationPanel(⟨CL11⟩)**Aufgabe:**

Steuert den gesamten GUI-Aufbau, initialisiert und koordiniert die Unterkomponenten, validiert Eingaben und sammelt Nutzerdaten für die Simulation.

Attribute:

keine

Operationen:

- `select_text_or_shape()`: Ermöglicht die Auswahl einer vordefinierten Formationsform oder Texteingabe.
- `validate_text()`: Prüft, ob ein gültiger Text oder eine gültige Auswahl vorliegt.
- `get_selected_text()`: Gibt den eingegebenen oder gewählten Text für die Formation zurück.

Kommunikationspartner:

GUI ((C10))

BackgroundPanel<CL12>

Aufgabe:

Auswahl des visuellen Hintergrunddesigns für die Simulation.

Attribute:

keine

Operationen:

select_background(): Gibt das ausgewählte visuelle Hintergrundthema zurück.

Kommunikationspartner:

GUI ((C10))

ImageUploadPanel<CL13>

Aufgabe:

Ermöglicht das Hochladen von Bilddateien zur Generierung von Bild-basierten Drohnenformationen.

Attribute:

keine

Operationen:

- `upload_image(): void` Öffnet einen Dateidialog und speichert die ausgewählte Bilddatei.
- `remove_image(): void` Entfernt ein zuvor geladenes Bild aus der Konfiguration.
- `show_preview(): void` Zeigt eine visuelle Vorschau des hochgeladenen Bilds im GUI-Bereich an.

Kommunikationspartner:

GUI ((C10))

SchedulePanel<CL14>

Aufgabe:

Verwaltung zeitlicher Abläufe für Text- oder Formationswechsel während der Simulation.

Attribute:

keine

Operationen:

- `generate_text_fields(count:int):` Erzeugt Eingabefelder zur Festlegung mehrerer Textwechsel inklusive Zeitpunkten.
- `get_schedule():` Gibt die eingeplanten Texte und ihre Zeitpunkte in strukturierter Form zurück.

Kommunikationspartner:

GUI ((C10))

DroneConfigPanel<CL15>

Aufgabe:

Verwaltung zeitlicher Abläufe für Text- oder Formationswechsel während der Simulation.

Attribute:

keine

Operationen:

- `select:drone_model()`: Gibt das aktuell ausgewählte Drohnenmodell zurück.
- `get_drone_count()`: Gibt die im Eingabefeld definierte Anzahl an Drohnen zurück.

Kommunikationspartner:

GUI ((C10))

ErrorDisplay<CL16>

Aufgabe:

Zeigt Fehlermeldungen und Hinweise für Benutzer:innen im GUI-Fenster an.

Attribute:

keine

Operationen:

`show_error(msg: str)`: Zeigt eine übergebene Fehlermeldung im GUI an.

`clear()`: Entfernt alle aktuell angezeigten Fehlermeldungen.

Kommunikationspartner:

GUI (C10)

5.2 C20 – ShowKonfigurator:

Die Komponente ShowKonfigurator ist das zentrale Modul zur Durchführung einer Drohnenshow. Sie übernimmt die Aufgabe, Nutzereingaben aus der GUI zu verarbeiten, Formationen zu laden, Drohnen zuzuweisen und die Simulation über PyBullet zu starten.

Technisch wird sie als eigenständige Python-Klasse realisiert, die mit anderen Modulen wie FormationsBibliothek, LichtEffektBibliothek und dem Projektmanagement interagiert. Die Simulationsschnittstelle erfolgt über Methodenaufrufe an die externe Bibliothek PyBullet.

5.2.1 Paket-/Klassendiagramm**ShowKonfigurator (CL20)****Aufgabe:**

Zentrale Instanz zur Steuerung der Showkonfiguration. Verbindet Benutzerkonfiguration mit Simulation, Szenen, Licht- und Formationslogik.

Attribute:

- user_input: dict – Benutzereingaben aus der GUI.
- simulator_connected: bool – Gibt an, ob PyBullet angebunden ist.
- drone_model_path: str – Pfad zur Drohnenmodell-Datei.

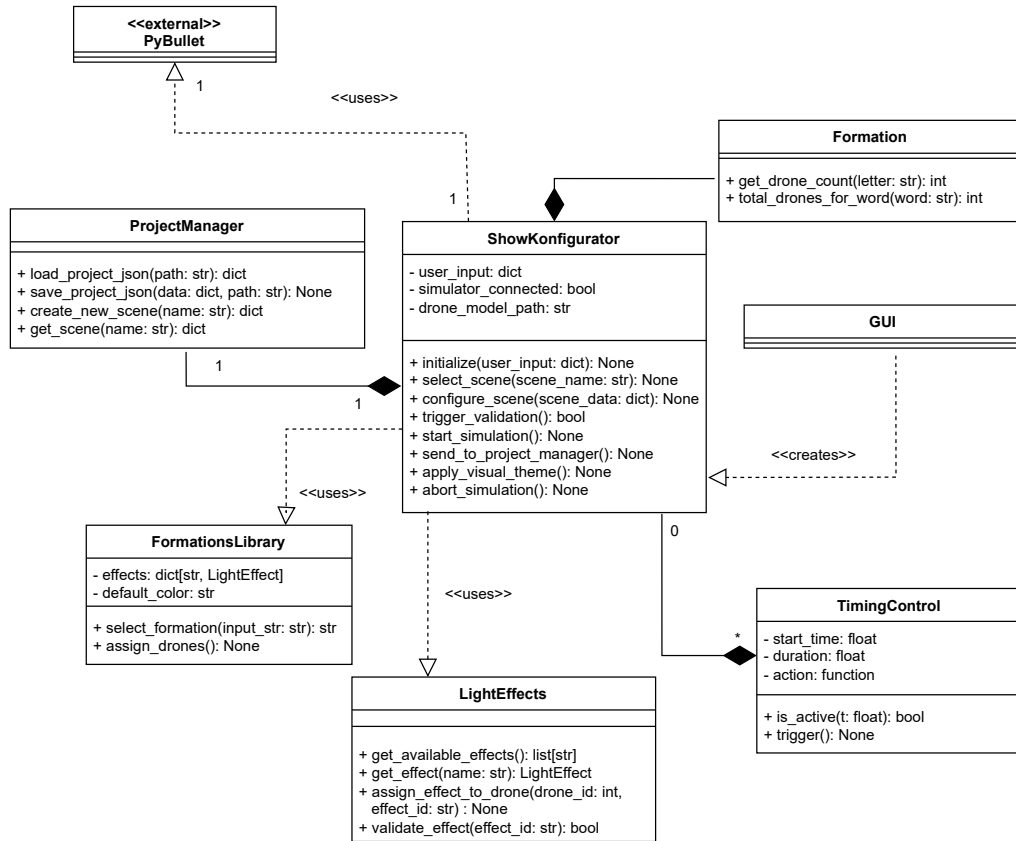


Abbildung 5.2: Klassendiagramm für Komponente ShowKonfigurator (C20)

Operationen:

- `initialize(user_input: dict)`: Initialisiert die interne Konfiguration mit Benutzereingaben.
- `select_scene(scene_name: str)`: Wählt eine Szene zur Bearbeitung.
- `configure_scene(scene_data: dict)`: Wendet Konfigurationsdaten auf die Szene an.
- `trigger_validation()`: Führt eine Validierung der Szene aus.
- `start_simulation()`: Startet die Simulation über PyBullet.
- `send_to_project_manager()`: Überträgt Konfiguration zur Projektspeicherung.
- `apply_visual_theme()`: Aktiviert optische Einstellungen (z.B. Hintergrundfarbe).
- `abort_simulation()`: Beendet die laufende Simulation.

Kommunikationspartner:

ProjectManager (⟨CL30⟩), FormationsLibrary (⟨CL21⟩), LightEffects (⟨CL22⟩), TimingControl (⟨CL23⟩), GUI (⟨CL10⟩), PyBullet (⟨External⟩)

FormationsLibrary ⟨CL21⟩**Aufgabe:**

Stellt Funktionen zur Auswahl und Zuweisung von Formationen bereit.

Attribute:

- `effects: dict[str, LightEffect]` – Sammlung aller Formationen nach ID.
- `default_color: str` – Standardfarbe für Drohnenformationen.

Operationen:

- `select_formation(input_str: str)`: Wählt eine Formation anhand der Eingabe.
- `assign_drones()`: Weist Drohnen einer Formation zu.

Kommunikationspartner:

ShowKonfigurator ((CL20))

LightEffects (CL22)

Aufgabe:

Verwaltet Drohnen-Lichteffekte.

Operationen:

- `get_available_effects()`: Gibt verfügbare Effekte zurück.
- `get_effect(name: str)`: Gibt spezifischen Effekt anhand des Namens zurück.
- `assign_effect_to_drone(drone_id: int, effect_id: str)`: Verknüpft Drohne mit Effekt.
- `validate_effect(effect_id: str)`: Prüft Gültigkeit eines Effekts.

Kommunikationspartner:

ShowKonfigurator ((CL20))

TimingControl (CL23)

Aufgabe:

Steuert zeitliche Abläufe während der Showausführung.

Attribute:

- `start_time`: float – Zeitpunkt des Beginns.
- `duration`: float – Dauer des Effekts oder Ereignisses.
- `action`: function – Funktion, die ausgeführt wird.

SEP2025

Drone Choreo "SkyOrchestrator"

Operationen:

- `is_active(t: float)`: Gibt zurück, ob das Event aktiv ist.
- `trigger()`: Führt die zugewiesene Aktion aus.

Kommunikationspartner:

ShowKonfigurator ($\langle \text{CL20} \rangle$)

5.3 C30 – Projektmanagement-Modul:

Die Komponente <C30> ist für das Laden, Speichern und Verwalten von Show-Konfigurationen zuständig. Sie kapselt alle Dateioperationen (Einlesen, Serialisieren, Validieren) für Konfigurationsdaten und stellt dafür strukturierte Schnittstellen bereit. Die Kommunikation erfolgt typischerweise über JSON-Dateien, die die vollständige Struktur einer Drohnenshow beschreiben.

5.3.1 Paket-/Klassendiagramm

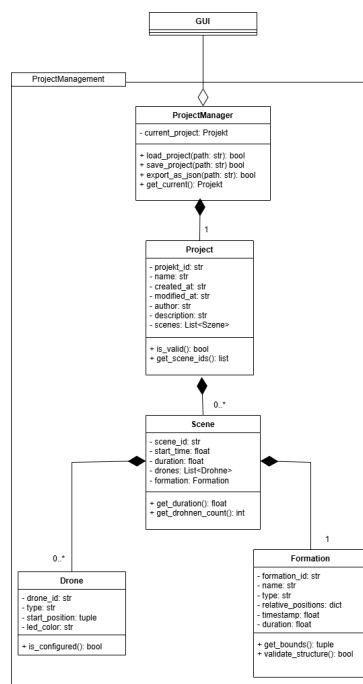


Abbildung 5.3: Klassendiagramm für Komponente <C30>

Beschreibung der Projektmanagement-Modul Klassen

ProjektManager<CL30>

Aufgabe:

Zentrale Verwaltung der Dateioperationen für Projekte. Koordiniert das Einlesen, Schreiben und Validieren von Projektdaten.

Attribute:

- `current_project`: Projekt aktuell geladene oder neu erstellte Show-Konfiguration

Operationen:

- `load_project(path: str)`: bool Lädt ein bestehendes JSON-Projekt von der Festplatte und erstellt ein entsprechendes Projekt-Objekt.
- `save_project(path: str)`: bool Serialisiert das aktuelle Projekt und speichert es als JSON-Datei.
- `export_as_json(path: str)`: bool – Exportiert die Konfiguration in ein universell verwendbares Format .
- `get_current()`: Projekt Gibt die aktuell verwaltete Projektinstanz zurück.

Kommunikationspartner:

GUI ((C10))

Projekt<CL31>

Aufgabe:

Repräsentiert ein vollständiges Drohnenshow-Projekt mit Szenen, Metadaten und struktureller Integrität.

Attribute:

- `projekt_id`: Eindeutige ID zur Projektidentifikation `name`: Name der Show-Konfiguration
- `created_at`: Zeitstempel zur Nachvollziehbarkeit von Erstellung und Bearbeitung
- `modified_at`: Zeitstempel zur Nachvollziehbarkeit von Erstellung und Bearbeitung
- `author`: Name der verantwortlichen Person oder Gruppe
- `description`: Freitextbeschreibung des Projektinhalts
- `scenes`: Alle zur Show gehörenden Szenen

Operationen:

- `is_valid()`: Validiert, ob das Projekt vollständig und konsistent aufgebaut ist.
- `get_scene_ids()`: Gibt eine Liste aller in der Show enthaltenen Szenen IDs zurück.

Kommunikationspartner:

ProjektManager(⟨CL30⟩), Szene (⟨CL32⟩)

Szene⟨CL32⟩

Aufgabe:

Modelliert eine einzelne Phase der Show inklusive Drohnenpositionen und Formation.

Attribute:

- `scene_id`: Eindeutige Kennung der Szene
- `start_time`: Zeitpunkt im Showablauf, zu dem die Szene beginnt
- `duration`: Dauer der Szene in Sekunden
- `drones`: Liste aller Drohnen in dieser Szene
- `formation`: Zugehörige Formation, die die Drohnen anordnet

Operationen:

- `get_duration()`: Gibt die geplante Dauer der Szene zurück.
- `get_drohnen_count()`: Zählt die Anzahl der enthaltenen Drohnen.

Kommunikationspartner:

Projekt(⟨CL31⟩), Drohne (⟨CL33⟩), Formation(⟨CL34⟩)

Drohne<CL33>

Aufgabe:

Repräsentiert eine Drohne mit konfigurierbaren Eigenschaften.

Attribute:

- drone_id: Interne ID der Drohne
- type: Bezeichnung des Drohnentyps
- start_position: Startkoordinate im Format (x, y, z)
- led_color: Initiale LED-Farbe der Drohne

Operationen:

- is_configured(): Gibt an, ob alle Pflichtangaben für die Drohne gesetzt wurden.

Kommunikationspartner:

Szene(<CL32>)

Formation<CL34>

Aufgabe:

Legt räumliche Anordnung und relative Koordinaten für Drohnen fest.

Attribute:

- `formation_id` : Technischer Bezeichner der Formation
- `name`: Menschlich lesbarer Name
- `type`: Typ der Formation (z.B. Kreis, Text, Bild)
- `relative_positions`: Dictionary mit Drohnenpositionen relativ zum Mittelpunkt
- `timestamp`: Zeitpunkt, zu dem die Formation eingenommen werden soll
- `duration`: Zeitraum, in dem die Formation gehalten werden soll

Operationen:

- `get_bounds()`: Berechnet die äußeren Grenzen der Formation.
- `validate_structure()`: Prüft, ob die Formation gültige Daten enthält (z.B. keine fehlenden Positionen).

Kommunikationspartner:

Szene(⟨CL32⟩)

5.4 C40 – FormationsBibliothek:

Die Komponente erzeugt neue Formationen (z. B. Kreise, Rechtecke, Texte), validiert diese gegen Kollisionen und speichert sie zentral ab. Benutzer:innen können über die GUI eine Formation auswählen oder eine individuelle Formation hochladen. Der ShowKonfigurator greift zur Laufzeit auf die gewählte Formation zu.

5.4.1 Paket-/Klassendiagramm

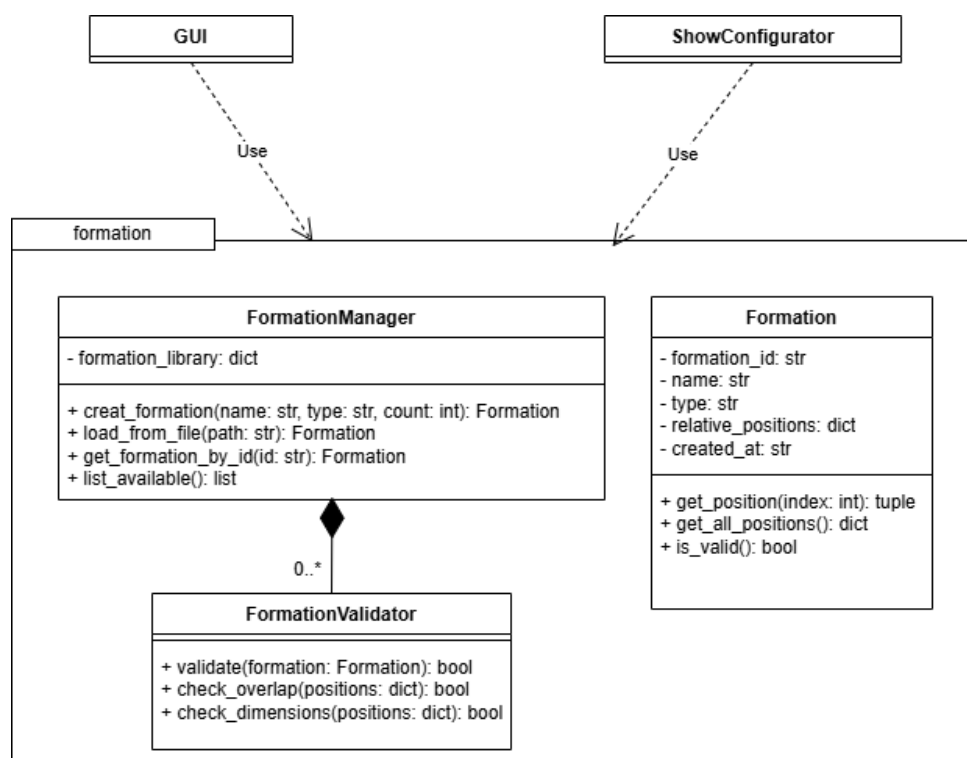


Abbildung 5.4: Klassendiagramm für Komponente <C40>

Beschreibung der Projektmanagement-Modul Klassen

FormationManager<CL40>

Aufgabe:

Zentrale Verwaltung der Dateioperationen für Projekte. Koordiniert das Einlesen, Schreiben und Validieren von Projektdaten.

Attribute:

- `formation_library`: Ein Wörterbuch, das Formationsobjekte mit ihrer eindeutigen ID verknüpft und zur internen Speicherung und schnellen Abfrage verwendet wird.

Operationen:

- `creat_formation(name, type, count)`: Erzeugt Formation anhand von Vorgabeparametern
- `load_from_file(path)`: Lädt Formation aus JSON- oder Textdatei
- `get_formation_by_id(id)`: Gibt Formation zur ID zurück
- `list_available()`: Gibt alle gespeicherten Formationen zur Auswahl zurück

Kommunikationspartner:

GUI ((C10)), ShowKonfigurator ((C20))

Formation CL41:

Aufgabe:

Repräsentiert eine Formation mit Positionsdaten und Metainformationen.

Attribute:

- `formation_id`: Eindeutige ID zur Identifikation der Formation
- `name`: Bezeichner zur Benennung durch den Benutzer
- `type`: Typ der Formation (z.B. 'circle', 'rectangle', 'text')
- `relative_positions`: Schlüssel-Wert-Paare, die Drohnenpositionen im relativen Koordinatensystem speichern
- `created_at`: Zeitstempel der Formationsgenerierung

Operationen:

- `get_position(index)`: Gibt Position für Drohne `i` zurück
- `get_all_positions()`: Gibt alle Koordinaten der Formation zurück
- `is_valid()`: Prüft auf strukturelle Konsistenz

Kommunikationspartner:

GUI (`<C10>`)

FormationValidator`<CL42>`

Aufgabe:

Validiert Formationen nach geometrischen und logischen Kriterien.

Attribute:

keine

Operationen:

- `validate(formation)`: Führt Gesamtprüfung durch
- `check_overlap(positions)`: Prüft auf sich überlappende Positionen
- `check_dimensions(positions)`: Prüft auf Einhaltung von Formgrößen oder Abmessungen

Kommunikationspartner:

FormationManager (`<CL40>`)

5.5 C50 – LightEffectLibrary:

Die Lichteffektbibliothek bietet eine zentralisierte Sammlung an Lichteffekten. Diese können über ID oder Namen referenziert werden und werden einzelnen Drohnen bzw. Formationen über Zeitslots zugewiesen. Dabei ist sichergestellt, dass alle Effekte validiert sind, bevor sie gespeichert oder zur Simulation weitergeleitet werden.

5.5.1 Paket-/Klassendiagramm

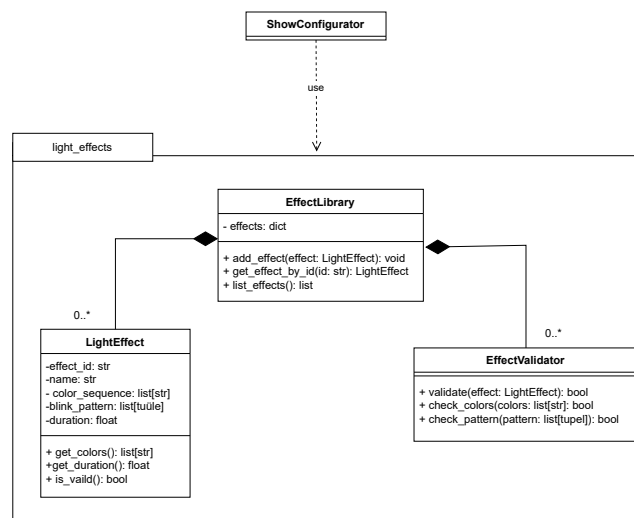


Abbildung 5.5: Klassendiagramm für Komponente $\langle C40 \rangle$

Beschreibung Lichteffektbibliothek Klassen

EffectLibrary: (<CL 51>)

Aufgabe:

Verwalten, Abrufen und Hinzufügen von Lichteffekten

Attribute:

effects: Schlüssel-ID-Zuordnung zu Lichteffekt-Objekten

Operationen:

- add_effect() Fügt einen neuen Effekt der Bibliothek hinzu.
- get_effect_by_id() Gibt einen Effekt anhand seiner ID zurück.
- list_effects() Gibt eine Liste aller registrierten Effekte zurück.

Kommunikationspartner:

ShowConfigurator (<C20>)

LightEffect CL52:

Aufgabe:

Definiert einen einzelnen LED-Lichteffekt mit Ablaufstruktur

Attribute:

- effect_id: Eindeutige Kennung
- name: Benutzerdefinierter Name
- color_sequence: Liste von Farbwerten
- blink_pattern: Zeitlich gesteuerte Blinkmuster (Zeit, Status)
- duration: Gesamtdauer des Effekts in Sekunden

Operationen:

- `get_colors()`: Gibt die definierten Farben zurück
- `get_duration()`: Gibt die Dauer des Effekts zurück
- `is_valid()`: Prüft ob Datenstruktur plausibel ist

Kommunikationspartner:

EffectLibrary ($\langle \text{CL51} \rangle$)

EffektValidator: $\langle \text{CL53} \rangle$

Aufgabe:

Validiert Lichteffektdaten auf strukturelle und semantische Korrektheit

Attribute:

keine

Operationen:

- `validate()` Vollständige Prüfung eines Effekts
- `check_colors()` Prüft ob Farbwerte korrekt formatiert sind
- `check_pattern()` Prüft ob Blinkmuster plausibel und sortiert sind

Kommunikationspartner:

EffectLibrary ($\langle \text{CL 51} \rangle$)

5.6 C70 – Validierungskomponente

Die Komponente Validierungskomponente übernimmt die Überprüfung der aktuellen Szene auf Vollständigkeit, Korrektheit und Plausibilität. Sie sammelt dabei Fehler in einer internen Liste und stellt diese der GUI zur Verfügung, sodass gezielt auf Probleme hingewiesen werden kann. Die Validierung greift dabei auf die Szenenverwaltung zu, um aktuelle Daten zu prüfen.

5.6.1 Paket-/Klassendiagramm

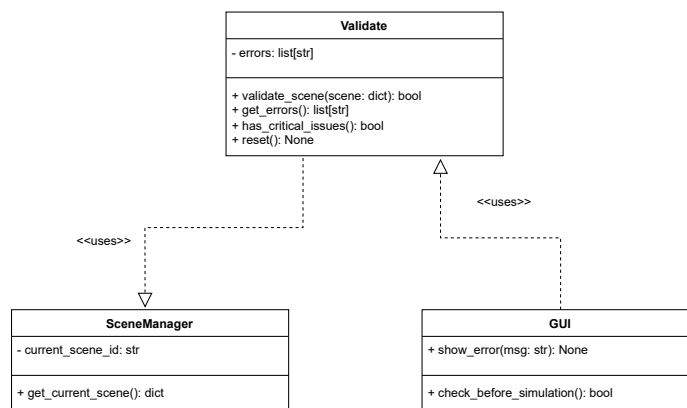


Abbildung 5.6: Klassendiagramm für Komponente C70

Validate <CL70>**Aufgabe:**

Zentrale Klasse zur Durchführung der Validierung von Szenen. Prüft Struktur, Inhalte und speichert gefundene Fehler intern.

Attribute:

- errors: list[str] – Liste aller erkannten Validierungsprobleme.

Operationen:

- validate_scene(scene: dict): bool – Führt die Validierung für die übergebene Szene durch und speichert dabei gefundene Fehler.
- get_errors(): list[str] – Gibt die gespeicherten Fehlermeldungen zurück.
- has_critical_issues(): bool – Prüft, ob schwerwiegende Probleme vorliegen, die eine Simulation verhindern würden.
- reset(): None – Löscht die gespeicherten Fehler aus der aktuellen Sitzung.

Kommunikationspartner:

SceneManager (<CL71>), GUI (<CL72>)

SceneManager <CL71>**Aufgabe:**

Stellt die aktuelle Szene zur Verfügung, die durch die Validierung geprüft wird.

Attribute:

- current_scene_id: str – Aktuell gewählte Szene-ID.

Operationen:

- `get_current_scene()`: dict – Gibt die vollständige Datenstruktur der aktuell aktiven Szene zurück.

Kommunikationspartner:

Validate (<CL70>)

GUI <CL72>

Aufgabe:

Empfängt Validierungsergebnisse und stellt Fehler benutzerfreundlich dar.

Attribute:

keine

Operationen:

- `show_error(msg: str)`: None – Gibt einen Fehlertext im GUI-Bereich aus.
- `check_before_simulation()`: bool – Prüft anhand der Validierungsergebnisse, ob eine Simulation möglich ist.

Kommunikationspartner:

Validate (<CL70>)

5.7 C80 – Datensysteme:

C80 speichert systemweit relevante Daten persistent. Dazu gehören Einstellungen der Nutzeroberfläche (z. B. letzte Datei, Sprache), Systemlogs zur Diagnose und optional exportierbare Inhalte. Die Ablage erfolgt in einem plattformneutralen Format (JSON oder CSV) und kann später durch Datenbankschnittstellen ersetzt werden.

5.7.1 Paket-/Klassendiagramm

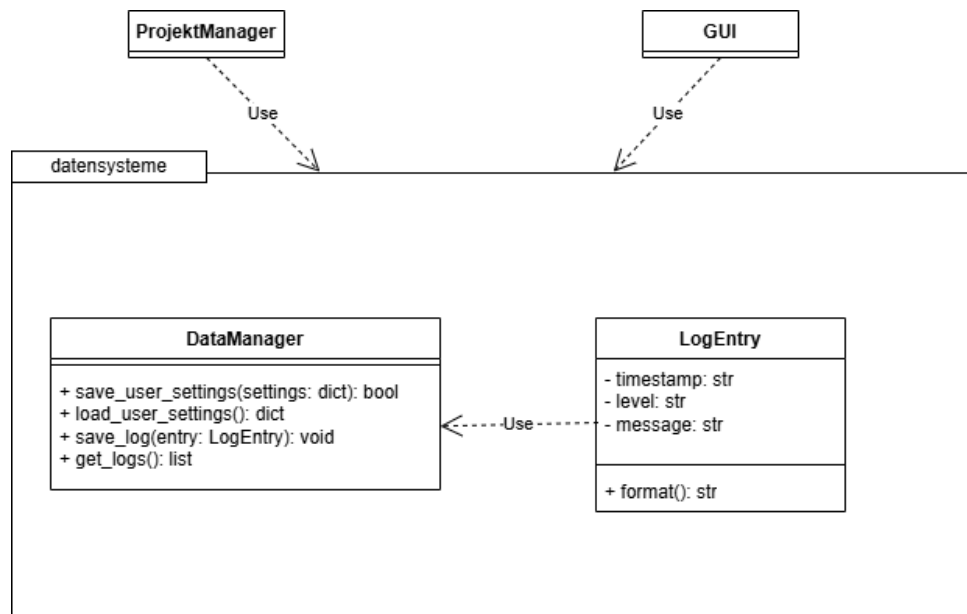


Abbildung 5.7: Klassendiagramm für Komponente $\langle C80 \rangle$

Beschreibung Datensysteme Klassen

DataManager (<CL 81>)

Aufgabe:

Koordiniert alle Dateioperationen der Komponente

Attribute:

keine

Operationen:

- `save_user_settings()` Speichert nutzerspezifische Einstellungen
- `load_user_settings()` Lädt gespeicherte Nutzereinstellungen
- `save_log(entry: LogEntry)` Persistiert einen Log-Eintrag
- `get_logs()` Gibt eine Liste aller bisherigen Logs zurück

Kommunikationspartner:

ProjektManager (<C30>) kann Logs oder Statusinfos speichern

GUI (<C10>) speichert Benutzerpräferenzen

LogEntry (<CL 82>)

Aufgabe:

Repräsentiert einen einzelnen Eintrag im Systemprotokoll

Attribute:

- timestamp: Zeit des Eintrags
- level: Log-Level (INFO, WARNING, ERROR)
- message: Inhalt der Meldung

Operationen:

- format(): Gibt den formatierten Log-Text zurück

Kommunikationspartner:

DataManager (<CL81>)

5.8 C90 – Szenenverwaltung

Die Komponente Szenenverwaltung dient der zentralen Organisation und Manipulation von Szenen innerhalb des Systems. Sie stellt Funktionen zur Auswahl, Bearbeitung und Verwaltung von Formationen und Effekten bereit. Weitere Module wie die Validierungskomponente nutzen ihre Funktionen, um vollständige Szenendaten zur Prüfung zu erhalten.

5.8.1 Paket-/Klassendiagramm

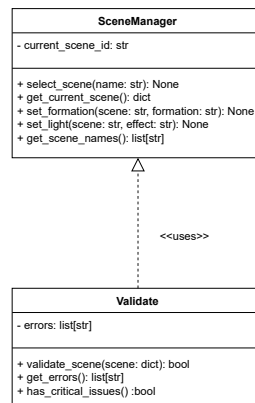


Abbildung 5.8: Klassendiagramm für Komponente Szenenverwaltung (C90)

SceneManager (CL90)

Aufgabe:

Verwaltet alle verfügbaren Szenen, erlaubt die Auswahl, Konfiguration und Abfrage aktueller Szeneninformationen.

Attribute:

- `current_scene_id`: str – Bezeichner der aktuell ausgewählten Szene.

Operationen:

- `select_scene(name: str)`: Aktiviert die Szene mit dem übergebenen Namen.
- `get_current_scene()`: Gibt die aktuell ausgewählte Szene als Dictionary zurück.
- `set_formation(scene: str, formation: str)`: Weist einer Szene eine Formation zu.
- `set_light(scene: str, effect: str)`: Verknüpft einen Lichteffekt mit einer Szene.
- `get_scene_names()`: Gibt eine Liste aller vorhandenen Szenennamen zurück.

Kommunikationspartner:

Validate (`<CL70>`)

6 Datenmodell

Die Anwendung SkyOrchestrator speichert die zentralen Informationen über geplante Shows, konfigurierte Drohnen, deren individuelle Flugbahnen, Formationen sowie durchgeführte Simulationen dauerhaft in einer dokumentenorientierten Datenbank. Die langfristig zu speichernden Daten werden in die Entitäten Show (E10), Drohne (E20), Flugbahn (E30), Formation (E40), Simulation (E50) und Wegpunkt (E60) aufgeteilt.

Ein Show-Objekt enthält Metadaten zur Drohnenshow und ist über eindeutige IDs mit den zugehörigen Drohnen, Formationen und Simulationen verknüpft. Jede Drohne besitzt eine eigene Flugbahn, die wiederum eine Liste von Wegpunkten beschreibt.

6.1 Diagramm

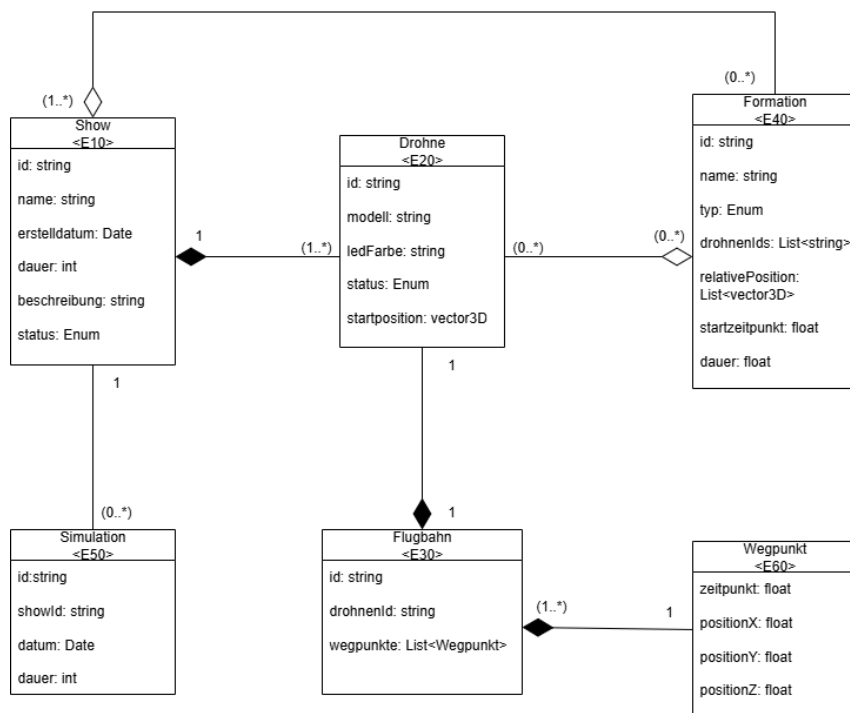


Abbildung 6.1: Klassendiagramm Produktdaten

6.2 Erläuterung

In den folgenden Tabellen wird die Beziehung zwischen den Entitäten Show $\langle E10 \rangle$, Drohne $\langle E20 \rangle$, Flugbahn $\langle E30 \rangle$, Formation $\langle E40 \rangle$, Simulation $\langle E50 \rangle$ und Wegpunkt $\langle E60 \rangle$ ausführlich dargestellt. Die Tabellen geben Aufschluss über die Art der Beziehung, die jeweilige Kardinalität, die erwartete Datenmenge sowie eine kurze inhaltliche Beschreibung. Grundlage der Modellierung sind die funktionalen Anforderungen an das System SkyOrchestrator.

Show $\langle E10 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 1–300 Drohnen je Show, pro Drohne ca. 1–2 KB	Einer Show können beliebig viele Drohnen zugeordnet werden. Jede Drohne gehört eindeutig zu einer Show.
Simulation	0..*	ca. 1–10 Simulationen, pro Show ca. 500–800 Byte	Eine Show kann mehrfach simuliert werden. Simulationen sind eindeutig einer Show zugeordnet.
Formation	0..*	ca. 0–100 Formationen, pro Show ca. 500–1000 Byte	Optional können Shows mehrere Formationen enthalten. Eine Formation beschreibt die gleichzeitige Anordnung mehrerer Drohnen.

Drohne $\langle E20 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	ca. 100–200 Byte	Jede Drohne ist genau einer Show zugeordnet.
Flugbahn	1	ca. 2–10 KB je Flugbahn	Jede Drohne besitzt genau eine Flugbahn, die als Liste von Wegpunkten gespeichert wird.
Formation	0..*	ca. 100–300 Byte Referenzdaten	Drohnen können in mehreren Formationen gleichzeitig oder nacheinander verwendet werden.

Flugbahn $\langle E30 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1	ca. 100–1000 Wegpunkte, gesamt ca. 2–10 KB	Die Flugbahn besteht aus einer Liste zeitlich geordneter Wegpunkte mit Informationen zu Position, Zeit, Bewegung und Lichtsteuerung.
Wegpunkt	1..*	ca. 100–1000 Wegpunkte	Eine Flugbahn besteht aus mehreren Wegpunkten, die den zeitlichen Verlauf bestimmen..

Formation $\langle E40 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 500–1000 Byte je Formation	Eine Formation umfasst mehrere Drohnen, deren Positionen relativ zur Formation angegeben werden.
Show	1..*	ca. 100–200 Byte	Jede Formation gehört eindeutig zu einer Show.

Simulation $\langle E50 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	ca. 500–800 Byte je Simulation	Eine Simulation ist eindeutig einer Show zugeordnet. Sie enthält u.a. Zeitstempel, Dauer, Kollisionserkennung und Energieverbrauch.

Wegpunkt $\langle E60 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Flugbahn	1	ca. 20–100 Byte je punkt	Eine Wegpunkt beschreibt eine konkrete Position der Drone zu einem bestimmten Zeitpunkt innerhalb ein Flugbahn z.b : Position (x, y, z), Zeit.

7 Konfiguration

In diesem Kapitel werden die technischen Voraussetzungen und Konfigurationsschritte beschrieben, die für die Installation, Ausführung und Nutzung von SkyOrchestrator erforderlich sind. Dazu zählen unter anderem die Systemvoraussetzungen, externe Abhängigkeiten sowie Hinweise zur Installation und zum Start der Anwendung.

7.1 Technische Voraussetzungen

Für die Ausführung von SkyOrchestrator wird eine lokale Python-Installation (Version 3.13 oder höher) sowie eine funktionierende Installation der Physik-Engine PyBullet benötigt, die für die dreidimensionale Darstellung und physikalische Simulation der Drohnenszenen verwendet wird. Darüber hinaus müssen weitere externe Python-Bibliotheken installiert werden, die in der mitgelieferten Datei `requirements.txt` aufgeführt sind. Diese befindet sich im Hauptverzeichnis von SkyOrchestrator und enthält folgende Abhängigkeiten:

- `numpy`
- `pybullet`
- `pandas`
- `scipy`

Diese Bibliotheken können über den folgenden Befehl im Terminal automatisch installiert werden:

```
python -m pip install -r requirements.txt
```

Hinweis: Im Verlauf der weiteren Entwicklung kann es dazu kommen, dass zusätzliche Bibliotheken benötigt werden. In diesem Fall wird die Datei `requirements.txt` entsprechend erweitert. Es empfiehlt sich daher, vor der Installation stets die aktuelle Version dieser Datei zu verwenden.

7.2 Projektstruktur

SkyOrchestrator ist modular aufgebaut und in verschiedene Verzeichnisse unterteilt, um die Wartbarkeit und Erweiterbarkeit der Software zu erleichtern. Im Folgenden wird ein Überblick über die wichtigsten Verzeichnisse und deren Funktionen gegeben:

- `main.py` – Startpunkt der Anwendung. Über diese Datei wird die grafische Benutzeroberfläche gestartet.
- `gui.py` – Quellcode der grafischen Benutzeroberfläche (GUI).
- `sim/` – Modul zur Durchführung der Simulationen mit PyBullet. Enthält die Funktionen zur Steuerung der Drohnenbewegungen.
- `models/` – Enthält die URDF Dateien der Dronen Modelle
- `savedProjects/` – Standardverzeichnis zur Speicherung von Benutzerszenen im `.json`-Format. Diese Dateien enthalten alle Konfigurationsdaten und können jederzeit erneut geladen werden.
- `requirements.txt` – Liste der externen Abhängigkeiten, die zur Ausführung des Programms erforderlich sind.
- `README.md` – Dokumentation mit Hinweisen zur Installation, Nutzung und zu häufig gestellten Fragen.

Diese Struktur stellt sicher, dass Benutzeroberfläche, Simulationslogik und Konfigurationsdaten klar voneinander getrennt sind. Dadurch wird nicht nur die Bedienung vereinfacht, sondern auch eine spätere Erweiterung des Systems erleichtert.

7.3 Start und Nutzung der Anwendung

SkyOrchestrator wird über die Datei `main.py` gestartet. Nach dem Wechsel in das Programmverzeichnis erfolgt der Start durch Eingabe des folgenden Befehls im Terminal:

```
python main.py
```

Nach dem Start öffnet sich das Hauptmenü der grafischen Benutzeroberfläche (GUI).

Der typische Arbeitsablauf im SkyOrchestrator beginnt mit dem Anlegen eines Projekts und einer oder mehrerer Szenen. Nach Eingabe der grundlegenden Metadaten (z.B. Name, Dauer,

Drohnenanzahl) werden Formationen, Bewegungen und Effekte konfiguriert. Die fertige Szene kann gespeichert, geladen, weiterbearbeitet und schließlich simuliert werden. Die einzelnen Schritte und Abläufe sind durch Zustands- und Sequenzdiagramme in den Kapiteln 1 und 2 detailliert beschrieben.

Die Simulation wird mit Hilfe von `PyBullet` dargestellt.

7.4 Konfigurationsdateien

SkyOrchestrator verwendet zur Speicherung und Wiederverwendung von Projekten sogenannte Sitzungsdateien im `.json`-Format. Diese Dateien werden standardmäßig im Unterordner `savedProjects` des SkyOrchestrator-Verzeichnisses abgelegt und können beim Speichern frei benannt werden. Der Zweck dieser Dateien besteht darin, sämtliche Einstellungen und Parameter einer Drohnenshow dauerhaft zu sichern und zu einem späteren Zeitpunkt wieder laden zu können.

Die in der `.json`-Dateien gespeicherten Parameter spiegeln die in Kapitel 2 beschriebenen Konfigurationsschritte wider. Dazu zählen insbesondere:

- **Metadaten der Szene:** Name, Gesamtdauer, Erstellungsdatum, Autor
- **Drohnenanzahl:** Anzahl der für die Szene vorgesehenen Drohnen
- **Formationsdaten:** Ausgewählte Formationen und deren zeitliche Abfolge
- **Bewegungsabläufe:** Zuordnung von Bewegungen zu einzelnen Drohnen oder Gruppen
- **Licht- und Zeiteffekte:** Art, Farbe, Zeitpunkt, Dauer und betroffene Drohnen
- **Timeline-Events:** Steuerung von Start, Ende, Formationswechseln und weiteren Effekten

Alle Parameter werden ausschließlich über die grafische Benutzeroberfläche eingestellt (siehe Abschnitt 4.4) und in der Datei gespeichert. Eine manuelle Bearbeitung der Konfigurationsdateien außerhalb der Anwendung ist nicht vorgesehen und wird nicht unterstützt.

Weitere Konfigurationsdateien, etwa zur Steuerung der Anwendung oder zur Systemkonfiguration, sind nicht erforderlich.

7.5 Konfiguration über die Benutzeroberfläche

SkyOrchestrator ermöglicht es dem Nutzer, sämtliche Einstellungen einer Drohnenshow komfortabel über die grafische Benutzeroberfläche vorzunehmen. Alle relevanten Steuerparameter – wie Drohnenanzahl, Formationen, Lichtanimationen, Zeitabläufe und weitere Szenenparameter – können individuell angepasst werden. Die zentrale Timeline-Komponente erlaubt es, Start- und Endzeitpunkte, Formationswechsel, Lichteffekte und Landungen präzise zu planen und zu steuern.

Während der laufenden Simulation bietet die Anwendung zusätzliche Funktionen, die die Interaktion und Kontrolle über den Ablauf erleichtern. Dazu zählen die freie Kamerasteuerung, mit der die Szene aus unterschiedlichen Perspektiven betrachtet werden kann, sowie die Möglichkeit, die Show über die Timeline zu verschiedenen Zeitpunkten zu analysieren. Außerdem kann die Simulation jederzeit vorzeitig abgebrochen werden, um Anpassungen vorzunehmen oder einen neuen Simulationsdurchlauf zu starten.

Alle grundlegenden Planungs- und Simulationsfunktionen sind so gestaltet, dass sie ohne Programmierkenntnisse bedient werden können. Eine Szene kann zu jedem Zeitpunkt gespeichert werden, unabhängig vom aktuellen Konfigurationsstand. Alle Eingaben werden vor der Simulation auf Gültigkeit geprüft. Die Anwendung prüft jedoch vor dem Start der Simulation, ob alle Pflichtfelder ausgefüllt und alle Eingaben gültig sind (siehe Zustandsdiagramm zur Szenenkonfiguration, Abbildung 1.3). Fehlerhafte oder unvollständige Szenen werden durch verständliche Hinweise kenntlich gemacht und können erst nach Korrektur simuliert werden.

7.6 Mögliche Erweiterungen

Zukünftige Versionen von SkyOrchestrator könnten um eine zentrale Konfigurationsdatei ergänzt werden, in der globale Parameter wie Drohnenanzahl, Startpositionen oder Lichtfarben dauerhaft gespeichert werden. Dies würde die Wiederverwendbarkeit und Portabilität erhöhen sowie die Trennung von grafischer Benutzeroberfläche und Konfigurationslogik verbessern.

Ebenfalls denkbar ist die Erweiterung um benutzerdefinierte Presets, die als Vorlagen für neue Shows dienen, sowie eine Versionsverwaltung für Projekte. Auch die Integration von Profileinstellungen, um verschiedene Nutzerpräferenzen zu unterstützen, wäre eine sinnvolle Erweiterung.

Diese Funktionen sind zum aktuellen Zeitpunkt noch nicht umgesetzt, werden jedoch als sinnvolle Ergänzungen für eine zukünftige Weiterentwicklung in Erwägung gezogen.

8 Änderungen gegenüber Fachentwurf

Im technischen Entwurf von *SkyOrchestrator* wurden gegenüber dem Fachentwurf folgende wesentliche Änderungen und Erweiterungen vorgenommen:

- **Einführung der Validierungskomponente (C70):**

Eine separate Komponente zur strukturierten Eingabeprüfung wurde hinzugefügt. Sie prüft die Szeneneingaben auf Vollständigkeit und Plausibilität und gibt strukturierte Fehlermeldungen an die grafische Oberfläche zurück.

- **Bild-zu-Formation-Arbeitsablauf:**

Es wurde ein neuer Prozess zur Konvertierung von Bilddateien in Drohnenformationen implementiert. Der Prozess ist mittels eines eigenen Aktivitätsdiagramms (siehe Abschnitt 1.2.3) festgehalten.

- **Modularisierung der grafischen Benutzeroberfläche (GUI):**

Die grafische Oberfläche wurde in spezialisierte Teilkomponenten wie `BildUploadBereich`, `DrohnenKonfigurationsPanel` und `ZeitplanPanel` gegliedert.

- **Erweiterung des Datenmodells durch Wegpunkte (E60):**

Die Flugbahnen einzelner Drohnen bestehen aus klar definierten Wegpunkten, die neben der Position auch Lichtparameter und Zeitinformationen enthalten. Mit dieser detaillierten Modellierung wird die ursprünglich vereinfachte Struktur erweitert.

- **Konsolidierte Schnittstellenspezifikation:**

Im technischen Entwurf wurden sämtliche Schnittstellen (z. B. zwischen grafischer Oberfläche, Szenenkonfigurator und weiteren Modulen) vollständig spezifiziert – inklusive Methoden, Parametern und Rückgabewerten.

- **Ergänzung neuer Zustands- und Ablaufdiagramme:**

Für zentrale Komponenten wie den Szenenkonfigurator und das Projektverwaltungsmodul wurden zusätzliche Zustandsdiagramme und Ablaufdarstellungen ergänzt, um die internen Abläufe zu verdeutlichen.

- **Einführung persistenter Formations- und Effektbibliotheken:**

Die Formationsbibliothek (C40) und die Lichteffektbibliothek (C50) wurden als wiederverwendbare, modular erweiterbare Komponenten umgesetzt.

- **Erweiterte Beschreibung der Konfiguration:**

Die Beschreibung der Installation, der Projektstruktur, des Anwendungsstarts sowie der Konfigurations- und Exportformate wurde umfassend ergänzt. Auch die Verwendung von JSON-Dateien zur Projektspeicherung wurde dokumentiert.

9 Erfüllung der Kriterien

Dieser Abschnitt dokumentiert, wie die Anforderungen aus dem Pflichtenheft durch die technische Umsetzung erfüllt wurden. Die Zuordnung erfolgt zu den jeweiligen Komponenten des Systems.

9.1 Musskriterien

- **RM1:** Simulation basiert auf Nutzereingaben
Erfüllt durch: ShowKonfigurator (C20), Simulationsbibliothek (C60)
- **RM2:** Visuelle Konfiguration von Formationen und Bewegungen
Erfüllt durch: GUI (C10), Formationsbibliothek (C40)
- **RM3:** Zeitsteuerung durch Timeline-Komponente
Erfüllt durch: ZeitplanPanel in der GUI (C10), Projektverwaltung (C30)
- **RM4:** Bedienung ohne Programmierkenntnisse
Erfüllt durch: intuitive GUI (C10), Validierungskomponente (C70)
- **RM5:** Fehlermeldungen bei ungültigen Eingaben
Erfüllt durch: Validierungskomponente (C70), Fehlermeldungsanzeige in GUI (C10)
- **RM6:** Lauffähigkeit unter Windows 10/11
Erfüllt durch: technische Basis mit Python 3.13, getestet unter Windows 11
- **RM7:** Anwendung startet im Hauptmenü ohne Fehler
Erfüllt durch: GUI-Initialisierung (C10), Projektmanager (C30)
- **RM8:** Projekte speichern und wiederladen
Erfüllt durch: Projektmanagement-Modul (C30), Datensysteme (C80)

9.2 Sollkriterien

- **RS1:** Steuerung von Lichtfarben und Helligkeit während der Simulation
Erfüllt durch: Lichteffektbibliothek (C50), ZeitplanPanel (C10)
- **RS2:** Unterstützung größerer Drohnenformationen (z. B. 100 Drohnen)
Erfüllt durch: Simulationsbibliothek (C60) mit realistischem Testfall
- **RS3:** Export in ein Austauschformat (JSON)
Erfüllt durch: Projektmanager (C30)
- **RS4:** Echtzeitnahe, flüssige 3D-Simulation
Erfüllt durch: Simulationsbibliothek auf Basis von PyBullet (C60)
- **RS5:** Optimierung der GUI auf Standardbildschirmauflösungen
Erfüllt durch: Responsive GUI (C10)
- **RS6:** Konfigurierbare Optionen (z. B. Sprache, Anzeigepräferenzen)
Teilweise erfüllt durch: DataManager (C80) – Speichern von Nutzerpräferenzen

9.3 Kannkriterien

- **RC1:** Import-/Exportoptionen für reale Shows
Teilweise erfüllt durch: Projektmanagement (C30), JSON-Exportfunktion
- **RC2:** Kollisionserkennung
Nicht umgesetzt (bewusst laut Abgrenzungskriterien RW2)
- **RC3:** Vordefinierte Bewegungsmuster / Vorlagen
Teilweise erfüllt durch: Formationsbibliothek (C40) mit einfachen Standardformationen
- **RC4:** Erweiterbare Systemarchitektur
Erfüllt durch: modulare Komponentenstruktur in C10-C90
- **RC5:** Musiksynchronisation
Nicht umgesetzt im Prototyp, aber als Erweiterung vorgesehen
- **RC6:** Einstellbare Geschwindigkeit der Drohnen
Nicht explizit umgesetzt
- **RC7:** Anzeige von Statusdaten einzelner Drohnen
Teilweise erfüllt durch: Drohnenstatusanzeige in GUI-Logik

10 Glossar

SkyOrchestrator: Software zur Planung, Konfiguration und Simulation von Drohnenshows mit visuellen Effekten in einer 3D-Umgebung.

Grafische Benutzeroberfläche (GUI): Oberfläche zur Steuerung aller Funktionen wie Szenenerstellung, Formationen, Effekte und Simulation.

Szene: Eine abgeschlossene Phase der Show mit definierten Drohnenformationen, Lichteffekten und zeitlichem Ablauf.

Formation: Räumliche Anordnung mehrerer Drohnen (z. B. Text, Herz, Bild, Kreis), die synchron gesteuert werden.

Lichteffekt: Farbige LED-Steuerung der Drohnen, z. B. Dauerlicht, Blinken oder Farbwechsel.

Timeline-Komponente: Zeitstrahl innerhalb der grafischen Oberfläche zur Planung von Startzeiten, Formationswechseln und Effekten.

Projekt: Gesamtheit mehrerer Szenen einer Show, gespeichert als JSON-Datei mit allen Metadaten.

PyBullet: Eingesetzte Physik-Engine zur realitätsnahen Simulation von Drohnenbewegungen in 3D.

ShowKonfigurator: Zentrale Steuerungseinheit der Anwendung, die GUI, Effekte, Projektlogik und Simulation verknüpft.

Projektmanagement-Modul: Verantwortlich für das Laden, Speichern und Exportieren von Projekten.

Formationsbibliothek: Sammlung vordefinierter Formationen; erweiterbar um benutzerdefinierte Anordnungen.

Lichteffektbibliothek: Verwaltung von Lichteffekten, die einzelnen oder mehreren Drohnen zugewiesen werden.

Validierungskomponente: Prüft Szeneneingaben auf Vollständigkeit, Korrektheit und Plausibilität.

Simulation: Echtzeit-Visualisierung der Show mit Drohnenbewegungen und Lichteffekten.

Wegpunkt: Konkrete Position, die eine Drohne zu einem bestimmten Zeitpunkt mit Licht- und Bewegungsparametern ansteuert.

URDF (Unified Robot Description Format): XML-basiertes Format zur Modellierung von Drohnenkörpern für die Simulation.

JSON (JavaScript Object Notation): Datenformat zur strukturierten Speicherung von Projekt- und Szenendaten.

Anwendungsfalldiagramm: UML-Diagramm zur Beschreibung der Funktionen eines Systems aus Nutzersicht.

Aktivitätsdiagramm: UML-Darstellung von Prozessen, Abläufen und Entscheidungslogik.

SEP (Softwareentwicklungspraktikum): Praktikumsveranstaltung der TU Braunschweig, in der das Projekt umgesetzt wurde.

TDSE (Technologietag der Studierendenprojekte): Veranstaltung zur Präsentation aller SEP-Projekte.